Quantum Computing_Lecture_Note

Quantum Computing 课堂笔记与翻译

Lecture Transcript: Administrative Matters & Quantum vs. Conventional Computing Review

Administrative Announcements

Class Attendance & Resources

Mohammad checked attendance for students who missed the previous class. He confirmed two students were present online.

He clarified the location of course resources:

- Primary and most recent resources are shared on **Microsoft Teams**.
- Materials are gradually migrated to **Blackboard**.
- The Quantum Computing channel on Teams is located under the **"hidden"** section. Students should navigate there if they cannot see it.

Session Recordings & Software Setup

- The recording of the first lecture session is available for students who missed it.
- Students **must install the required software framework** before proceeding.
  - A Python setup is assumed to be already in place.
  - The specific requirement is to install **Squishkit**.
- The programming session will begin once installations are complete.

Academic Content Review

Core Difference: Computing Stacks

Mohammad initiated a review by asking about the fundamental difference between quantum and conventional computing stacks.

> 🙋‍♂️ **Student**: What is the difference? 👨‍🏫 **Mohammad**: The **conventional computing stack** depends on binary bits (**0 and 1**) and is based on **Newtonian (classical) physics**. In contrast, the **quantum computing stack** is based on **quantum physics**.

He noted that detailed explanations were covered in the first session and should be reviewed via the recording.

Problems for Conventional vs. Quantum Computers

The discussion moved to types of computational problems. Mohammad listed problems that are challenging for conventional computers but are targets for quantum computing:

1. **Factoring large integers** (a given example).
2. **Searching an unsorted database**.
3. **Simulating quantum systems**.
4. **Optimization problems** (encompassing many specific issues).
5. **Machine Learning and Artificial Intelligence** problems.

He reminded students of a task from the previous class: to identify other problems that conventional computing stacks cannot solve effectively.

讲座记录：行政事务与量子计算 vs. 传统计算回顾

行政通知

课堂出勤与资源

Mohammad 核对了上一节课缺席学生的出勤情况，确认有两名学生在线出席。

他明确了课程资源的位置：

- 主要和最新的资源分享在 **Microsoft Teams** 上。
- 材料会逐步迁移到 **Blackboard**。

- Teams 上的量子计算频道位于 **"hidden"（隐藏）** 部分。如果看不到，学生应导航至该处查找。

课程录制与软件安装

- 第一讲的录制内容已提供，供缺席的学生补看。
- 学生**必须安装所需的软件框架**才能继续。
  - 假定学生已具备 Python 环境。
  - 具体要求是安装 **Squishkit**。
- 软件安装完成后，编程环节将开始。

学术内容回顾

核心差异：计算栈

Mohammad 通过提问量子计算栈与传统计算栈的根本区别来开始复习。

> 🙋‍♂️ **学生**：区别是什么？ 🤦‍♂️ **Mohammad**：**传统计算栈**依赖于二进制位（**0 和 1**），并基于**牛顿（经典）物理学**。相比之下，**量子计算栈**基于**量子物理学**。

他指出，详细解释已在第一讲中涵盖，应通过录制内容复习。

传统计算机与量子计算机面临的问题

讨论转向了计算问题的类型。Mohammad 列举了传统计算机难以解决、但量子计算旨在攻克的问题：

1. **大整数因式分解**（已给出的例子）。
2. **搜索未排序的数据库**。
3. **模拟量子系统本身**。
4. **优化问题**（包含许多具体问题）。
5. **机器学习和人工智能**领域的问题。

他提醒学生上一节课布置的一项任务：找出其他传统计算栈无法有效解决的问题。

Lecture Content: Quantum Computing Problems & Qubit Hardware

Mohammad continues listing problems that are difficult for conventional computers but are targets for quantum computing:

1. **Searching an unsorted database**.
2. **Simulating quantum systems**.
3. **Optimization problems** (which encompass many specific issues).
4. **Problems in machine learning and AI**.

He then revisits a task assigned in the previous lecture, asking students to research two key areas:

1. Identify other problems beyond those listed that are hard for conventional computing stacks.
2. Investigate the **hardware design of qubits**. Specifically, he asks what underlying hardware enables qubit design and which techniques are currently leading the field.

He provides examples of hardware platforms for qubits, such as **superconducting qubits**, trapped ions, quantum dots, and nuclear magnetic resonance. He encourages students interested in hardware to research which approach is currently dominant.

Recap: Qubit vs. Bit & A Conceptual Experiment

Mohammad initiates a review by asking about the fundamental difference between a bit and a qubit.

> 🙋‍♂️ **Speaker 2**: 0, 1 or both? 👨‍🏫 **Mohammad**: Yes. A bit can be 0 *or* 1 at one time, but a qubit can simultaneously be 0 *and* 1 until you measure it. Until measurement, it is in a superposition of both states.

To illustrate how a qubit can be conceptualized in hardware, **Mohammad** describes a simplified experimental setup using optics.

Experimental Setup for a Photonic Qubit:

- **Source**: A **single-photon source** that emits one photon at a time.
- **Component**: A **switchable mirror** that can be toggled between being transparent or reflective.

Demonstrating a Classical Bit (0 or 1):

1. With the switchable mirror set to **reflective**, an incoming photon bounces off and is detected at a specific location, which we define as representing the state `1`.
2. With the mirror set to **transparent**, the photon passes through and is detected at a different location, representing the state `0`.
3. In this configuration, the photon's path (and thus the bit value) is determined by the fixed state of the mirror *before* the photon is emitted.

讲座内容：量子计算问题与量子比特硬件

Mohammad 继续列举对传统计算机困难但属于量子计算目标的问题：

1. **搜索未排序的数据库**。
2. **模拟量子系统**。
3. **优化问题**（包含许多具体问题）。
4. **机器学习和人工智能领域的问题**。

随后，他回顾了上一节课布置的一项任务，要求学生研究两个关键领域：

1. 找出除已列出问题外，其他对传统计算栈而言困难的问题。
2. 研究**量子比特的硬件设计**。具体来说，他询问是什么底层硬件使得量子比特设计成为可能，以及目前哪些技术在该领域处于领先地位。

他列举了实现量子比特的硬件平台示例，例如**超导量子比特**、囚禁离子、量子点和核磁共振。他鼓励对硬件感兴趣的学生去研究当前哪种方法占主导地位。

回顾：量子比特 vs. 比特 & 一个概念性实验

Mohammad 通过询问比特和量子比特之间的根本区别来开始回顾。

> 🙋‍♂️ **学生2**: 0, 1 还是两者都是？ 🤵 **Mohammad**: 是的。一个比特在某一时刻可以是 0 *或* 1，但一个量子比特在测量之前可以同时是 0 *和* 1。在测量之前，它处于两种状态的叠加态。

为了说明如何在硬件中概念化一个量子比特，**Mohammad** 描述了一个使用光学的简化实验装置。

光子量子比特的实验装置：

- **源**：一个**单光子源**，每次发射一个光子。
- **组件**：一个**可切换镜**，可以在透明和反射状态之间切换。

演示经典比特（0 或 1）：

1. 当可切换镜设置为**反射**状态时，入射光子被反射并在特定位置被检测到，我们将其定义为表示状态 `1`。
2. 当镜子设置为**透明**状态时，光子直接穿过并在不同位置被检测到，表示状态 `0`。
3. 在此配置中，光子的路径（即比特值）由光子发射*之前*镜子的固定状态决定。

Quantum Computing: From Classical Bits to Qubits

Demonstrating a Classical Bit with a Switchable Mirror

If the photon is detected at the designated location, we assume it represents a `1`. This is how we demonstrate a classical bit with our hardware setup.

When the **switchable mirror** is set to **reflective**, it behaves like a reflective mirror. The photon hits it, reflects, and is detected at the `1` location. Conversely, if we set the mirror to **transparent**, the photon passes through and is captured at a different location, representing a `0`. This demonstrates the encoding of `0` and `1`.

This principle is the foundation of technologies like **optical fiber communication**, where `0`s and `1`s are transmitted using similar phenomena of light guiding.

Conceptual Demonstration of a Qubit with a Half-Silvered Mirror

Now, consider the same single-photon source, but this time with a **half-silvered mirror**. When a photon arrives, some photons will reflect, and some will pass through. Consequently, at any given moment, there is a probability of detecting the photon at *both* the `0` and `1` locations.

> 👩‍🏫 Lecturer: The key point is that we do not actually build qubits this way. This setup is purely to conceptually demonstrate, at first glance, how `0` and `1` can be present simultaneously. It's to show the physical possibility.

This concept is initially hard to digest—how can `0` and `1` both be available at the same time? The demonstration shows that with a half-silvered mirror, light can be detected at both the `0` and `1` positions, meaning both states are probabilistically present. This simple setup illustrates that `0` and `1` can coexist, which is the core idea behind a qubit's **superposition**.

However, this technology needs significant refinement. **This is not the method that yields the desired results expected in quantum computing**; it is merely a conceptual demonstration of possibility.

Wave-Particle Duality and Experimental Context

In our last class, I showed a video demonstrating **wave-particle duality**. The video addressed the question: Is light a wave or a particle?

Historically, light was initially considered a sort of particle. However, **Thomas Young** demonstrated through his **double-slit experiment** that light also exhibits wave-like behavior. We already knew light could behave as a particle.

> 👩‍🏫 Lecturer: So, it's a kind of **duality**. The crucial point is that light's behavior depends on the **experimental setup** you design. Sometimes light behaves as a wave, and sometimes it behaves as a particle. That means it exhibits both properties.

量子计算：从经典比特到量子比特

使用可切换镜演示经典比特

如果光子在指定位置被检测到，我们假设它代表 `1`。这就是我们如何用硬件设置演示一个经典比特。

当**可切换镜**设置为**反射**状态时，它的行为就像一面反射镜。光子击中它，发生反射，并在 `1` 的位置被检测到。相反，如果我们将镜子设置为**透明**状态，光子会直接穿过并在另一个位置被捕获，代表 `0`。这演示了 `0` 和 `1` 的编码。

这一原理是**光纤通信**等技术的基础，其中 `0` 和 `1` 就是利用类似的光导现象进行传输的。

使用半镀银镜对量子比特进行概念性演示

现在，考虑同一个单光子源，但这次使用**半镀银镜**。当一个光子到达时，一些光子会反射，一些会透射。因此，在任何给定时刻，都有可能在 `0` 和 `1` 两个位置检测到光子。

> 👩‍🏫 讲师：关键点在于，我们实际上并不是这样构建量子比特的。这个设置纯粹是为了从概念上初步演示 `0` 和 `1` 如何能够同时存在。这是为了展示物理上的可能性。

这个概念起初很难理解——`0` 和 `1` 怎么可能同时可用？这个演示表明，使用半镀银镜，光可以在 `0` 和 `1` 两个位置被检测到，这意味着两种状态都以概率形式存在。这个简单的设置说明了 `0` 和 `1` 可以共存，这正是量子比特**叠加态**背后的核心思想。

然而，这项技术需要重大改进。**这并不是能产生量子计算所期望结果的方法**；它仅仅是对可能性的一种概念性演示。

波粒二象性与实验背景

在上节课中，我展示了一个演示**波粒二象性**的视频。视频探讨了一个问题：光是波还是粒子？

历史上，光最初被认为是一种粒子。然而，**托马斯·杨**通过他的**双缝实验**证明，光也表现出类似波的行为。我们已经知道光可以表现为粒子。

> 👩‍🏫 讲师：所以，这是一种**二象性**。关键点在于，光的行为取决于你设计的**实验装置**。有时光表现为波，有时表现为粒子。这意味着它具有两种属性。

Wave-Particle Duality and Quantum Concepts

Through the double-slit experiment, Thomas Young demonstrated that light is a wave. However, light was already known to be a particle. This creates a **duality**.

The key point is that the behavior of light depends on the **experimental setup** you design. Depending on that setup, light can behave as a wave or as a particle. This means it possesses both properties. Light is both a wave and a particle, depending on how you observe it.

The outcome depends on the **measurement**. It depends on how you measure it and how you set up the experiment. That setup will demonstrate light sometimes as a wave and sometimes as a particle. This is the essence of duality.

Connecting Duality to Qubits

Why discuss this duality? We are applying this exact concept of duality when designing **qubits**. In a qubit, there is a duality: it is both 0 and 1 until you measure it. This is a form of the duality concept.

Our habitual understanding of nature is not dualistic; we see it as concrete, where results are final. These quantum concepts challenge that worldview. We must be adaptable and broaden our understanding to grasp such ideas.

Review: Quantum vs. Conventional Physics

In the last class on quantum physics, we discussed two concepts unique to it. Can you recall them? We discussed the differences between quantum and conventional physics.

In conventional physics, phenomena are **deterministic**. Any metric you observe will take values—either discrete or continuous values. In contrast, quantum physics is not deterministic; it is **probabilistic**. Secondly, metrics in quantum physics take **discrete values**.

Apart from these foundational differences, what two unique properties distinguish quantum physics? One was the **superposition** effect, and the other was **entanglement**.

Understanding Superposition

Superposition is precisely the concept where two things exist at exactly the same time. In one view, both 0 and 1 coexist—until the moment you measure it. That is the concept of superposition, and it is only possible in quantum physics.

At one place, two things can coexist. Consider this: in our world, at any specific point, place, and time, can two things coexist simultaneously? You would likely say no, it's not possible. This challenges our classical intuition.

波粒二象性与量子概念

通过双缝实验，托马斯·杨证明了光是一种波。然而，光也早已被确认为是一种粒子。这就形成了一种**二象性**。

关键在于，光的行为取决于你设计的**实验装置**。根据不同的装置，光可以表现为波，也可以表现为粒子。这意味着它同时具备两种属性。光既是波也是粒子，这取决于你观察它的方式。

结果取决于**测量**。它取决于你如何测量以及如何设置实验。该设置将有时把光展示为波，有时展示为粒子。这就是二象性的本质。

将二象性连接到量子比特

为什么要讨论这种二象性？我们在设计**量子比特**时，运用的正是这种二象性概念。在一个量子比特中，存在二象性：在测量之前，它既是0也是1。这是一种二象性概念。

我们对自然的习惯性理解不是二元的；我们认为自然是具体的，结果是确定的。这些量子概念挑战了这种世界观。我们必须适应并拓宽我们的理解以掌握这些概念。

回顾：量子物理与传统物理

在上节关于量子物理的课程中，我们讨论了其独有的两个概念。你们能回想起来吗？我们讨论了量子物理与传统物理之间的差异。

在传统物理中，现象是**确定性的**。你观察到的任何度量都会取值——要么是离散值，要么是连续值。相比之下，量子物理不是确定性的；它是**概率性的**。其次，量子物理中的度量取**离散值**。

除了这些基本差异之外，哪两个独特属性区分了量子物理？一个是**叠加**效应，另一个是**纠缠**。

理解叠加

叠加正是两个事物同时存在的概念。从一种视角看，0和1是共存的——直到你测量它的那一刻。这就是叠加的概念，并且它只在量子物理中才可能。

在一个地点，两个事物可以共存。思考一下：在我们的世界中，在任何一个特定的点、地点和时间，两个事物能同时共存吗？你可能会说不，这不可能。这挑战了我们经典的直觉。

The Nature of Measurement and Superposition

In our conventional world, at any specific point, place, and time, two things cannot coexist simultaneously. From the lens of conventional physics, this is impossible. However, in **quantum physics**, it is possible.

At the micro level, two things can coexist at the same time—but with a crucial caveat: *until you measure it*. When you measure it, you will see only one thing. Yet, before your measurement, the things were coexisting. This is the concept of **superposition**.

We must leverage **superposition** to perform quantum computing. The key point is that we cannot perceive superposition directly because nature has not equipped us with the necessary senses to experience this effect. However, its absence from our perception does not mean it does not exist in nature.

The Limits of Human Senses and Measurement

Measurement, in essence, is an extension of our senses. Whether using our five senses directly or tools like glasses, microscopes, or telescopes, we are merely enhancing our existing sensory capacities. The fundamental sensors remain the same.

> 🙋 Student: The things are a little complex. If we cannot see that same thing when we're measuring, looking at the same place, then how are we coming to the conclusion that it exists? 👩‍🏫 Lecturer: It's based on the concept. Suppose the concept of **superposition** exists, but you can't perceive it with your own or extended senses. However, when you apply this concept for computational purposes in quantum computing—operating on the *assumption* that superposition exists—and you design your hardware based on that notion… if the hardware you build then functions correctly, it validates the underlying concept. This demonstrates that our senses are not sufficient to delve into that depth to realize it directly.

测量与叠加的本质

在我们的经典世界中，在任何特定的点、地点和时间，两个事物不能同时共存。从经典物理学的视角来看，这根本不可能。然而，在**量子物理学**中，这是可能的。

在微观层面，两个事物可以同时共存——但有一个关键的警告：*直到你测量它为止*。当你测量时，你只会看到其中一个事物。但在你测量之前，事物是共存的。这就是**叠加**的概念。

我们必须利用**叠加**来进行量子计算。关键在于，我们无法直接感知叠加，因为大自然没有赋予我们必要的感官来体验这种效应。然而，它不在我们的感知范围内，并不意味着它在自然界中不存在。

人类感官与测量的局限

从本质上讲，测量是我们感官的延伸。无论是直接使用我们的五种感官，还是使用眼镜、显微镜或望远镜等工具，我们都只是在增强现有的感官能力。基础的传感器仍然是相同的。

> 🙋 学生：事情有点复杂。如果我们在测量时、看着同一个地方，却无法看到那个相同的东西，那么我们是如何得出它存在这个结论的呢？ 👨‍🏫 讲师：这是基于概念。假设**叠加**这个概念存在，但你无法用自己或延伸的感官感知到它。然而，当你为了量子计算的计算目的应用这个概念时——基于*假设*叠加存在——并依据这个观念设计你的硬件……如果你构建的硬件随后能正确运行，这就验证了其底层概念。这表明我们的感官不足以深入到那个层面去直接认识它。

Conceptual Validation through Application

If you design hardware based on the *assumed* concept of **superposition**, and that hardware then functions correctly for computation, it validates the underlying concept. This demonstrates that our senses are insufficient to perceive that depth of reality directly, but the concept solves the intended purpose. This is the current state of quantum computing.

The concept is mathematically valid and proven. If you design hardware based on it, you can use it for computational purposes and it will function well. The key point is that our five senses, even with extended capabilities, cannot access that part of nature hidden from them.

Historical Shift from Theory to Application

This was the historical challenge: the concept existed but lacked an applied component. When applied computing demonstrated that hardware built on these concepts could perform computations, perception shifted positively. Without this applied proof, the ideas were often dismissed. The move to practical application made people accept quantum phenomena as real.

Lecture & Lab Objectives

> 👨‍🏫 Lecturer: Any other question? Can you all download this first? And we will then compute it.

To understand quantum computing, we must understand some of its mathematics. Mathematics is the language we use to write and understand concepts. In this or the next lecture, we will deal with complex numbers.

> 👨‍🏫 Lecturer: Anyone knows what are the complex numbers? Complex numbers? IJ. Yeah, that's IOTA thing. It's not so difficult to understand, but at least one lecture I need to demonstrate the language of quantum physics.

Lab Aims

In this lab, our aims are:

1. Learn to build a simple quantum circuit.
2. See how the **superposition** concept is realized using a circuit.
3. Run a circuit on a simulator and interpret its result.
4. Explore the concept of **entanglement** and see how to realize it.

Administrative Tasks

- **Install Qiskit**: If you have not installed Qiskit, please install it. You can just Google "install Qiskit"; it's very simple.
- **Visualization**: In physics, visualization has a big impact. We will visualize how qubits behave and how to manipulate them, to see where your end result is.

完整中文翻译

通过应用进行概念验证

如果你基于*假设*的**叠加**概念设计硬件，并且该硬件随后能正确运行以进行计算，这就验证了底层概念。这表明我们的感官不足以直接感知那个深度的现实，但该概念解决了预期目的。这就是量子计算当前的状况。

这个概念在数学上是有效且经过证明的。如果你基于它设计硬件，你可以将其用于计算目的，并且它会运行良好。关键在于，我们的五种感官，即使借助扩展能力，也无法触及对它们隐藏的那部分自然。

从理论到应用的历史性转变

这正是历史上的挑战：概念存在但缺乏应用部分。当应用计算证明基于这些概念构建的硬件能够执行计算时，人们的看法才发生了积极的转变。没有这种应用证明，这些想法常常被否定。向实际应用的转变使人们开始接受量子现象是真实的。

讲座与实验目标

👩‍🏫 讲师：还有其他问题吗？大家能先下载这个吗？然后我们将进行计算。

要理解量子计算，我们必须理解其部分数学。数学是我们用来书写和理解概念的语言。在本节或下节课中，我们将处理复数。

> 👩‍🏫 讲师：有人知道什么是复数吗？复数？IJ。是的，就是那个IOTA（虚数单位）。理解起来并不那么难，但至少我需要用一节课来演示量子物理的语言。

实验目标

本次实验，我们的目标是：

1. 学习构建一个简单的量子电路。
2. 了解如何使用电路实现**叠加**概念。
3. 在模拟器上运行电路并解释其结果。
4. 探索**纠缠**概念，并了解如何实现它。

行政任务

- **安装 Qiskit**：如果您尚未安装 Qiskit，请安装它。您可以直接谷歌搜索"install Qiskit"；这非常简单。
- **可视化**：在物理学中，可视化影响很大。我们将可视化量子比特的行为以及如何操纵它们，以查看最终结果所在。

课程内容与可视化工具

接下来，您将在模拟器上运行一个电路并解释其结果。然后，我们将探讨**纠缠**的概念，并了解如何实现它。

在物理学中，可视化影响很大。您需要可视化最终结果的位置。为了可视化量子比特的行为以及如何操纵它们，有一个名为 **Blocksphere** 的工具，我们也会学习一下它。

量子计算的发展阶段与电路设计

今天我将使用一些**量子门**。在量子计算中，也存在门。我今天会告诉你们这些门的作用，但具体它们如何实现这些功能，我将在后续阶段讲解。至少我会告知你们，如果在电路中应用这些门，具体会发生什么。

在我上一堂课中，我讲述了量子计算目前所处的发展阶段。我们现在处于计算步骤的哪个阶段？我们现在非常接近硬件层面。想象一下，回到30到70年前，那时没有编程语言，只有硬件，你必须直接与硬件交互进行编码，甚至需要为此设计电路。

我们现在正是在这个层面上理解量子计算。我们必须设计电路来解决任何问题。无论出现什么问题，你都必须为其设计量子电路。目前，我可以简单地说它也是一个量子处理器，但请不要误解——这里的"处理器"意味着它将完成所有工作。这是一个用于执行特定任务的量子处理器。

例如，当我展示**叠加**效应，并说这是一个量子电路时，我也可以将其与"量子处理器"一词互换，即这是一个用于展示叠加效应的量子处理器。我们将在以后讨论通用处理器，等到硬件行为良好时，人们才会去开发通用的量子处理器。

目前，我们正处于一个阶段，需要从头开始构建每一个电路来解决任何特定问题。例如，如果我必须解决一个因数分解问题，我将设计一个仅能解决该因数分解问题的电路。与传统的计算栈相比，我目前所处的阶段并非"如果你要解决因数分解问题，就构建一个算法"的阶段。

> 🙋 学生：经典门？ 🤦 讲师：是的，经典门。在量子计算中，也有门。

Course Content and Visualization Tools

Next, you will run a circuit on a simulator and interpret its result. Then, we will explore the concept of **entanglement** and see how you can realize it.

In physics, visualization impacts a lot. You want to visualize where your end result is. For visualizing qubits, how qubits are playing their part and how you want to manipulate them, there is a tool called **Blocksphere**. We'll learn a bit about Blocksphere as well.

Stage of Quantum Computing & Circuit Design

Today I'm going to use some **quantum gates**. In quantum computing, there are also gates. I can tell you today what those gates will do, but *how* those gates do that kind of thing, I will tell you at a later stage. At least I will inform you what exactly happens if you apply these gates in your circuit.

In my last class, I told you about the current stage in quantum computing. We are in which stage right now in terms of the computing step? We are very near to hardware right now. Imagine and take yourself back 30 to maybe 60-70 years before. There was no programming language, nothing was there, and only hardware was there. You had to directly interact with the hardware for coding purposes, and even to the extent that in order to do something using hardware, you had to design the circuit for that.

We are understanding quantum computing on that level right now. We have to design the circuit to solve any problem. So whatever problem comes, you have to design the quantum circuit for it. For the time being, I can simply say that it is a quantum processor as well, but don't take it otherwise—that "processor" means the processor will do everything. This is a quantum processor to do a specific thing.

For example, when I'm showing a **superposition** effect and I'm saying that it is a quantum circuit for that, I can interchange it with the term "quantum processor"—that this is a quantum processor for showing the superpositioning effect. We will go to the generic processor later when everything is behaving well, like hardware is behaving well; then people will go for a generic quantum processor later.

Right now, we are in a stage where we are building from scratch each and every circuit to solve any specific problem. So if I have to solve a factorization problem, I will design a circuit which will only solve the factorization problem. I'm not in a stage right now, if you compare it from the conventional stack, where if you have to solve a factorization problem, you build an algorithm.

> 🙋‍♂️ Student: Classical gates? 👨‍🏫 Lecturer: Yeah. Classical gates? So, in quantum computing, there are also gates.

Current State of Quantum Computing

Right now, we are in a stage where we are building each and every circuit from scratch to solve any specific problem. For example, if I have to solve a **factorization problem**, I will design a circuit that will *only* solve that specific problem.

This is not like the conventional computing stack, where you build an algorithm, put it into a programming language, and a compiler automatically converts it to run on a generic processor. In quantum computing, you must resonate with a different concept: you know the entire hardware and the **gates**, which are the building blocks.

Problem-Solving in Quantum Computing

To solve a problem in quantum computing, we must first see and translate the problem into a **quantum-compatible** form. We do not directly solve problems in the habitual, algorithmic way humans think. First, you must translate the problem from the normal world into the quantum world.

This translation is crucial. The important aspect is that you must break down your problem statement to identify:

1. What are the input parameters?
2. How do you want to represent these inputs?

The Primacy of Qubit Representation

The first and foremost thing in quantum computing is problem representation. You must determine **how many qubits** will be required to represent the problem. This is the first step.

Currently, because **quantum bit** hardware design is so costly and maintaining its quantum state is expensive, the focus is on using the **minimum number of qubits**. An algorithm or circuit designer who can solve a problem with fewer qubits is considered to have created a good solution.

The Qubit Cost Analogy

This situation is analogous to the early days of classical computing when memory was very expensive. At that time, any solution had to factor in memory usage and aim to use less of it because memory was a costly resource.

Today, when designing classical algorithms, we hardly take care of memory because it is so cheap. The same concept applies in the quantum world right now. The **qubit** represents quantum memory, and this "memory" is currently quite expensive. Therefore, the goal is to **minimize the usage of this quantum memory** when designing your solution.

量子计算的当前状态

目前，我们正处于一个需要从头开始构建每一个电路来解决任何特定问题的阶段。例如，如果我必须解决一个**因式分解问题**，我将设计一个*仅能*解决该特定问题的电路。

这不同于传统的计算堆栈，在那里你构建一个算法，将其放入编程语言，然后编译器会自动将其转换以在通用处理器上运行。在量子计算中，你必须理解一个不同的概念：你了解整个硬件和作为构建模块的**量子门**。

量子计算中的问题解决

要在量子计算中解决问题，我们必须首先审视问题并将其转化为**量子兼容**的形式。我们不以人类习惯的、算法式的思维方式直接解决问题。首先，你必须将问题从正常世界翻译到量子世界。

这种翻译至关重要。其重要方面在于，你必须分解你的问题陈述以确定：

1. 输入参数是什么？
2. 你希望如何表示这些输入？

量子比特表示的首要性

量子计算中首要的事情是问题表示。你必须确定表示这个问题需要**多少个量子比特**。这是第一步。

目前，由于**量子比特**硬件设计成本极高，且维持其量子态非常昂贵，因此重点在于使用**最少数量的量子比特**。能够用更少量子比特解决问题的算法或电路设计者，被认为创造了一个好的解决方案。

量子比特成本的类比

这种情况类似于经典计算的早期，当时内存非常昂贵。那时，任何解决方案都必须考虑内存使用，并旨在使用更少的内存，因为内存是昂贵的资源。

如今，在设计经典算法时，我们几乎不考虑内存，因为它非常便宜。同样的概念目前也适用于量子世界。**量子比特**代表了量子内存，而这种"内存"目前相当昂贵。因此，在设计解决方案时，目标是**最小化这种量子内存的使用**。

设计量子算法的核心步骤

目前在设计任何算法时，我们几乎不考虑内存，因为当今世界的内存非常便宜。同样的概念目前也适用于量子世界。**量子比特**代表了量子内存，而这种内存目前相当昂贵。因此，你的目标是**最小化这种量子内存的使用**。

你应该以使用更少数量的量子比特的方式来设计你的算法。这是首要关注点。

问题分解与量子化

当你手头有一个问题时，首先需要将其分解，弄清楚输入是什么，以及如何使其与量子计算兼容。在这个过程中，首要任务是弄清楚需要多少量子比特来表示这个问题。

一旦确定了所需的量子比特数量，接下来就需要确定它们的**初始状态**。初始状态就是你开始计算时的状态。因为你是在为某个问题设计解决方案，所以你至少知道期望的答案是什么。

状态向量的表示

当你使用这些状态时，它是一种数学状态。你将用某种**数学状态向量**来表示你的输入状态。这是你的初始状态向量，而你的期望状态是最终状态向量。你必须完成一段从输入状态到输出状态的转换旅程。你知道输入状态，也知道输出状态。

量子电路的设计

此时，你的量子物理知识就派上用场了。在这两者之间，你将生成一个**量子电路**。你需要构思一个电路，它以初始状态为输入，并生成期望的最终状态作为输出。这就是你的量子电路或量子处理器。

具体实施过程

任何手头的问题，首先都要分解并使其量子兼容。你会确定需要 `n` 个量子比特来解决你的问题。在这 `n` 个量子比特上，你需要找出它们的初始状态。例如，这个量子比特可能处于 `|0>` 态，那个可能处于 `|1>` 态，依此类推。你需要弄清楚每个量子比特的初始状态。总的来说，这就是你的初始输入状态向量。

定义初始与最终状态

这就是你表示问题的方式。你必须以这些量子比特能够表示问题的方式来表述它，并且那就是它们的初始状态。现在你知道你的最终状态将是什么。同样是这些量子比特，你期望最终这个比特输出为 `1`，那个输出为 `0`，等等。这就是我的最终期望，我希望我的最终状态是这样的。关键在于，这就是我的答案。

> 🙋 学生：所以每个人都知道，如果你有一个输入并且需要解决某个问题，你知道答案，但你不知道是什么电路能给你带来那个答案？ 👨‍🏫 讲师：是的，正是在这中间，你…

设计量子算法的核心步骤

Currently, when designing any algorithm, we hardly consider memory because it is so cheap in today's world. The same concept applies to the quantum world right now. **Qubits** represent quantum memory, and this memory is quite expensive at present. Therefore, your goal is to **minimize the usage of this quantum memory**.

You should design your algorithm in such a way that it uses a smaller number of qubits. That is the primary focus.

Problem Decomposition and Quantization

When you have a problem at hand, first you need to break it down, figure out what the inputs are, and how to make it quantum-compatible. In this process, the first and foremost task is to determine how many qubits are needed to represent this problem.

Once the number of required qubits is determined, the next step is to identify their **initial state**. The initial state is the state from which you start the computation. Because you are designing a solution for a problem, you at least know what the expected answer is.

Representation of State Vectors

When using these states, it is a sort of mathematical state. You will represent your input state using a **mathematical state vector**. This is your initial state vector, and your desired state is the final state vector. You must undertake a transition journey from the input state to the output state. You know the input state, and you know the output state.

Design of Quantum Circuits

At this point, your knowledge of quantum physics comes into play. In between these states, you will generate a **quantum circuit**. You need to devise a circuit that takes the initial state as input and produces the desired final state as output. This is your quantum circuit or quantum processor.

Specific Implementation Process

For any problem at hand, first decompose it and make it quantum-compatible. You will determine that `n` number of qubits will solve your problem. On these `n` qubits, you need to find out their initial state. For example, this qubit may be in the `|0>` state, that one may be in the `|1>` state, and so on. You need to figure out the initial state of each and every qubit. In total, this is your initial input state vector.

Defining Initial and Final States

This is how you represent the problem. You must represent the problem in such a way that these qubits can represent it, and that will be their initial state. Now you know what your final state will be. Using the same number of qubits, you expect that, for instance, this bit should output `1`, that one should output `0`, and so on. This is my final expectation; I want my final state to be like this. The point is, this is my answer.

> 🙋 Student: So everyone knows that if you have an input and something to solve, you know the answer, but you don't know what circuit will bring you that answer? 👨 🏫 Lecturer: Yes, and here in between, you…

Quantum Circuit Design and Measurement

My final expectation is for the output state to be specific, for example, `|1⟩` or `|0⟩`. This is the answer I am aiming for.

The core challenge in quantum algorithm design is knowing the desired input and output, but not knowing the specific **quantum circuit** (or **quantum processor**) that transforms the input state into the final output state. The input state passes through this circuit to generate the output.

Measurement as Part of Algorithm Design

A critical part of designing a quantum algorithm is deciding **which qubits to measure** from the final output. I am only interested in specific qubits, so I need to measure those.

> 🙋 Student: Why is selecting which qubits to measure so important? 👨 🏫 Lecturer: Because before measurement, the state is **probabilistic**. The outcome is not certain; it exists in a **superposition**. You only get a concrete result (`0` or `1`) when you measure a qubit. Until then, you might expect a `1`, but it could collapse to a `0`.

Initialization and Basic Gates

In our circuit design, initialization typically sets all qubits to `|0⟩`. If you need a qubit to start as `|1⟩`, you must apply an **X-gate** (a **NOT gate**) to it.

In today's class, we will get a glimpse of several key quantum gates. We will primarily use the **H gate**, **X gate**, and **Z gate**.

- The **X gate** is a NOT gate: `|0⟩` becomes `|1⟩` and vice versa.
- The **Z gate** also performs a NOT-like operation but introduces a **phase difference** or **phase change**.
- Similarly, the **Y gate** is another NOT gate variant.

The **X, Y, and Z gates** are all types of NOT gates, but with a crucial distinction: the X gate does **not** introduce a phase difference, while the Y and Z gates **do** introduce different types of phase differences. We will discuss what "phase difference" means in more detail later.

Setting Up the Quantum Circuit

First, we need to perform certain imports to build our circuits. We use the `QuantumCircuit` class from Qiskit to construct quantum circuits.

量子电路设计与测量

我的最终期望是输出态是特定的，例如 `|1⟩` 或 `|0⟩`。这就是我想要得到的答案。

量子算法设计的核心挑战在于，知道期望的输入和输出，但不知道将输入态转换为最终输出态的具体**量子电路**（或**量子处理器**）。输入态通过这个电路来产生输出。

测量作为算法设计的一部分

设计量子算法的一个关键部分是决定从最终输出中**测量哪些量子比特**。我只对特定的量子比特感兴趣，因此我需要测量它们。

> 🙋‍♂️ 学生：为什么选择测量哪些量子比特如此重要？ 👨‍🏫 讲师：因为在测量之前，量子态是**概率性的**。结果是不确定的；它存在于**叠加态**中。只有当你测量一个量子比特时，你才会得到一个具体的结果（`0` 或 `1`）。在此之前，你可能期望得到 `1`，但它也可能坍缩为 `0`。

初始化与基本量子门

在我们的电路设计中，初始化通常将所有量子比特设置为 `|0⟩`。如果你需要一个量子比特以 `|1⟩` 开始，你必须对它施加一个 **X 门**（一个**非门**）。

在今天的课程中，我们将初步了解几个关键的量子门。我们将主要使用 **H 门**、**X 门**和 **Z 门**。

- **X 门** 是一个非门：`|0⟩` 变为 `|1⟩`，反之亦然。
- **Z 门** 也执行类似非门的操作，但会引入一个**相位差**或**相位变化**。
- 类似地，**Y 门** 是另一种非门变体。

X、Y 和 Z 门都是非门类型，但有一个关键区别：X 门**不**引入相位差，而 Y 和 Z 门**会**引入不同类型的相位差。我们稍后将更详细地讨论"相位差"的含义。

设置量子电路

首先，我们需要进行一些导入来构建我们的电路。我们使用 Qiskit 中的 `QuantumCircuit` 类来构建量子电路。

X 门不会引入任何相位差，而 **Y 门**和 **Z 门**则会引入某种相位差。关于相位差的具体含义，我们稍后再详细讨论。

设置量子电路：导入与工具

首先，我们需要进行一些导入来构建电路。我们使用 Qiskit 中的 `QuantumCircuit` 类来构建量子电路。安装 Qiskit 后，所有组件都会自动包含，你只需要导入它们即可。

当电路设计完成后，我们需要模拟它。为此，我将使用 **Aer 模拟器**。Aer 是 Qiskit 内置的模拟器，目前我们将主要使用它。当然，未来如果需要，我也会介绍其他模拟器。

为了进行可视化，我们还需要导入绘图工具。此外，为了理解量子状态，我们需要用到**态矢量**的概念。态矢量是一种数学语言，用于描述系统的初始状态。在数学上，它就是一个向量。

为了理解这些态矢量（其中涉及数学概念），我今天可能还会介绍复数。NumPy 和 Math 库大家应该都熟悉，请跟着我一起操作。

应用哈达玛门

现在，我们要做的第一件事是在一个单量子比特上应用一个**哈达玛门**。哈达玛门非常有趣。

假设你有一个量子比特，它处于初始的 |0〉状态。如果你对它应用一个哈达玛门，结果会是什么？结果是**叠加态**。在测量之前，|0〉和 |1〉两种状态会同时存在。

那么，在数学上我们如何表示 |0〉和 |1〉同时存在呢？初始状态是 |0〉，但应用哈达玛门后的状态是 (|0〉 + |1〉) / √2。

如果要用数学语言描述态矢量，它就是：`(1/√2) * |0〉 + (1/√2) * |1〉`

[中文翻译]

X 门不会引入任何相位差，而 **Y 门**和 **Z 门**则会引入某种相位差。关于相位差的具体含义，我们稍后再详细讨论。

设置量子电路：导入与工具

首先，我们需要进行一些导入来构建电路。我们使用 Qiskit 中的 `QuantumCircuit` 类来构建量子电路。安装 Qiskit 后，所有组件都会自动包含，你只需要导入它们即可。

当电路设计完成后，我们需要模拟它。为此，我将使用 **Aer 模拟器**。Aer 是 Qiskit 内置的模拟器，目前我们将主要使用它。当然，未来如果需要，我也会介绍其他模拟器。

为了进行可视化，我们还需要导入绘图工具。此外，为了理解量子状态，我们需要用到**态矢量**的概念。态矢量是一种数学语言，用于描述系统的初始状态。在数学上，它就是一个向量。

为了理解这些态矢量（其中涉及数学概念），我今天可能还会介绍复数。NumPy 和 Math 库大家应该都熟悉，请跟着我一起操作。

应用哈达玛门

现在，我们要做的第一件事是在一个单量子比特上应用一个**哈达玛门**。哈达玛门非常有趣。

假设你有一个量子比特，它处于初始的 |0〉状态。如果你对它应用一个哈达玛门，结果会是什么？结果是**叠加态**。在测量之前，|0〉和 |1〉两种状态会同时存在。

那么，在数学上我们如何表示 |0〉和 |1〉同时存在呢？初始状态是 |0〉，但应用哈达玛门后的状态是 (|0〉 + |1〉) / √2。

如果要用数学语言描述态矢量，它就是：`(1/√2) * |0〉 + (1/√2) * |1〉`

Quantum Bit (Qubit) State Representation

Initial State and Superposition

The initial state is `|0〉`. After applying a Hadamard gate, the state becomes a **superposition** of `|0〉` and `|1〉`. Mathematically, the state vector is: `(1/√2) * |0〉 + (1/√2) * |1〉`

General Qubit Representation

A qubit is a **superposition** of `|0〉` and `|1〉`. To represent a qubit, we use the general form: `α|0〉 + β|1〉` Here, `α` is the **amplitude** (or magnitude) associated with the `|0〉` state vector, and `β` is the **amplitude** associated with the `|1〉` state vector.

The Normalization Condition

A fundamental condition for a valid quantum state is that the sum of the squares of the amplitudes must equal 1: $\alpha^2 + \beta^2 = 1$ This condition arises because quantum mechanics is **probabilistic**. The square of an amplitude ($|\alpha|^2$ or $|\beta|^2$) gives the probability of the qubit collapsing to that corresponding state upon measurement.

Conceptual Understanding of Superposition

For a more intuitive understanding, you can think of a qubit as a **combination** of $|0\rangle$ and $|1\rangle$. This combination implies there is "some part" of $|0\rangle$ and "some part" of $|1\rangle$. These "parts" are precisely quantified by the amplitudes α and β.

Analogy: Wave Amplitude

The concept of amplitude in a qubit is analogous to the amplitude of a physical wave (e.g., a sound wave represented as a sinusoid). The amplitude of a wave is its peak value—the maximum height it attains from its baseline.

- Similarly, α is the amplitude corresponding to the $|0\rangle$ basis vector.
- β is the amplitude corresponding to the $|1\rangle$ basis vector.

Probability and Measurement

In quantum mechanics, everything is **probabilistic**. These amplitudes are directly linked to probability. When you measure a qubit, its superposition **collapses** to either the definite state $|0\rangle$ or $|1\rangle$.

- The probability of collapsing to $|0\rangle$ is $|\alpha|^2$.
- The probability of collapsing to $|1\rangle$ is $|\beta|^2$.

For example, if you measure the same qubit state 100 times, the number of times you get $|0\rangle$ versus $|1\rangle$ will be determined by these probabilities, which is why $\alpha^2 + \beta^2$ must equal 1 (representing a total probability of 100%).

量子比特（Qubit）的状态表示

初始状态与叠加态

初始状态是 `|0⟩`。应用哈德玛门后，状态变为 `|0⟩` 和 `|1⟩` 的**叠加态**。数学上，态矢量表示为：`(1/√2) * |0⟩ + (1/√2) * |1⟩`

通用的量子比特表示

一个量子比特是 `|0⟩` 和 `|1⟩` 的**叠加态**。为了表示一个量子比特，我们使用通用形式：`α|0⟩ + β|1⟩` 这里，`α` 是与 `|0⟩` 态矢量相关的**振幅**（或幅度），`β` 是与 `|1⟩` 态矢量相关的**振幅**。

归一化条件

一个有效量子态的基本条件是振幅的平方和必须等于 1：`α² + β² = 1` 这个条件源于量子力学的**概率性**。振幅的平方（`|α|²` 或 `|β|²`）给出了量子比特在测量时坍缩到对应状态的概率。

对叠加态的概念性理解

为了更直观的理解，你可以把量子比特看作是 `|0⟩` 和 `|1⟩` 的**组合**。这种组合意味着存在 `|0⟩` 的"一部分"和 `|1⟩` 的"一部分"。这些"部分"正是由振幅 `α` 和 `β` 精确量化的。

类比：波的振幅

量子比特中振幅的概念类似于物理波（例如，表示为正弦曲线的声波）的振幅。波的振幅是其峰值——从基线达到的最大高度。

- 类似地，`α` 是对应于 `|0⟩` 基矢的振幅。
- `β` 是对应于 `|1⟩` 基矢的振幅。

概率与测量

在量子力学中，一切都是**概率性的**。这些振幅直接与概率相关。当你测量一个量子比特时，它的叠加态会**坍缩**到确定的状态 `|0⟩` 或 `|1⟩`。

- 坍缩到 `|0⟩` 的概率是 `|α|²`。
- 坍缩到 `|1⟩` 的概率是 `|β|²`。

例如，如果你对同一个量子比特状态测量 100 次，得到 `|0⟩` 与 `|1⟩` 的次数将由这些概率决定，这就是为什么 `α² + β²` 必须等于 1（代表总概率为 100%）。

Probability and Measurement

The probability of a measurement outcome is directly associated with the **amplitude** of the quantum state. When you measure a qubit, its superposition **collapses** to either `|0⟩` or `|1⟩`.

- The probability of collapsing to `|0⟩` is `|α|²`.
- The probability of collapsing to `|1⟩` is `|β|²`.

For example, if you measure the same qubit state 100 times, the number of times you get `|0⟩` versus `|1⟩` is determined by these probabilities. This is why `α² + β²` must equal 1, representing a total probability of 100%.

Concrete Example: The Hadamard Gate

Let's return to the concrete example of applying a **Hadamard gate** to a `|0⟩` qubit.

> 🙋 Student: 1 by root 2. 👨‍🏫 Lecturer: 1 by root 2. What is beta? 1 by root 2. So 1 by root 2 squared is? 🙋 Student: 1 by 2. 👨‍🏫 Lecturer: 1 by 2. So 1 by 2 means, if you measure it 100 times, you'll get `|0⟩` 50 times and `|1⟩` 50 times. This is exactly a 50-50 superposition.

Algorithm Design and Hardware Gates

Depending on our algorithm design, we may need different probability distributions upon measurement. For instance, we might require 70% `|0⟩` and 30% `|1⟩`, or 90% `|0⟩` and 10% `|1⟩`.

- The **Hadamard gate** is a fundamental hardware-supported building block that provides equal 50-50 superposition.
- The task of quantum algorithm and circuit design is to combine or create other gates to achieve the specific probability distributions (`60%` / `40%`, `90%` / `10%`, etc.) required by the algorithm.
- As quantum engineers or algorithm designers, we must understand what each hardware gate (like the Hadamard) does. Applying a Hadamard gate in a circuit guarantees that upon measurement, the output will be `|0⟩` 50% of the time and `|1⟩` 50% of the time.

完整中文翻译

概率与测量

测量结果的概率直接与量子态的**振幅**相关。当你测量一个量子比特时，它的叠加态会**坍缩**到 `|0⟩` 或 `|1⟩`。

- 坍缩到 `|0⟩` 的概率是 `|α|²`。
- 坍缩到 `|1⟩` 的概率是 `|β|²`。

例如，如果你对同一个量子比特状态测量 100 次，得到 `|0⟩` 与 `|1⟩` 的次数将由这些概率决定，这就是为什么 `α² + β²` 必须等于 1（代表总概率为 100%）。

具体示例：哈达玛门

让我们回到对 `|0⟩` 量子比特应用**哈达玛门**的具体例子。

> 🙋 学生：1 除以根号 2。🤦 讲师：1 除以根号 2。那 beta 是多少？1 除以根号 2。那么 1 除以根号 2 的平方是？🙋 学生：1/2。🤦 讲师：1/2。所以 1/2 意味着，如果你测量 100 次，你会得到 `|0⟩` 50 次，`|1⟩` 50 次。这正是 50-50 的叠加态。

算法设计与硬件门

根据我们的算法设计，我们可能需要测量时产生不同的概率分布。例如，我们可能需要 70% 的 `|0⟩` 和 30% 的 `|1⟩`，或者 90% 的 `|0⟩` 和 10% 的 `|1⟩`。

- **哈达玛门** 是一个基本的、由硬件支持的基础构建模块，它提供均等的 50-50 叠加态。
- 量子算法和电路设计的任务就是组合或创建其他门，以实现算法所需的特定概率分布（`60%`/`40%`、`90%`/`10%` 等）。
- 作为量子工程师或算法设计者，我们必须理解每个硬件门（如哈达玛门）的作用。在电路中应用哈达玛门可以保证在测量时，输出有 50% 的概率是 `|0⟩`，50% 的概率是 `|1⟩`。

Quantum State Vectors and the Hadamard Gate

The Hadamard Gate's Function

At the hardware level, as a quantum engineer, you must understand what the **Hadamard gate** does. If you apply the Hadamard gate in your circuit, it will give you an **equal**

superposition, a 50-50% superposition. When you measure the output, 50% of the time it will be `|0⟩` and 50% of the time it will be `|1⟩`.

The Probability Constraint: $\alpha^2 + \beta^2 = 1$

Now, why must `α² + β²` equal 1? Why can it not be greater than or less than 1? The answer lies in **probability**. The total probability must always be 1.

- If `α² + β²` were less than 1 (e.g., 0.7), it would mean that out of 100 measurements, you would only get `|0⟩` or `|1⟩` a total of 70 times.
- What about the remaining 30 times? The output must be something, and the only possible outputs are `|0⟩` and `|1⟩`. There is no third option.

Therefore, `α² + β²` must equal 1. This is a fundamental check for a valid quantum state vector. If you are given any state vector, a simple way to verify its validity is to check if `α² + β² = 1`. If it is not 1, it is **not a valid state vector** in quantum computing.

Simulating Quantum States on a Classical Machine

State vectors like `|0⟩` are interpretable by humans. However, if you need to **simulate** a quantum circuit on a conventional machine, you must represent these state vectors programmatically. Simulation means writing a program to model the circuit's behavior from start to finish, including what each gate (like the Hadamard gate) does.

- Programming a quantum simulation is essentially a **game of vector manipulation**.
- You start with an initial state vector (e.g., `|0⟩`, represented as `α=1, β=0`).
- You then apply operations (gates) that manipulate this vector. For example, applying the Hadamard gate transforms the state.
- Even for complex circuits, the core principle remains **state vector manipulation**. You begin in a specific state and continuously manipulate it over time to achieve the final result.

量子态向量与哈达玛门

哈达玛门的功能

在硬件层面，作为一名量子工程师，你必须理解**哈达玛门**的作用。如果你在电路中应用哈达玛门，它将给你一个**均等叠加态**，即 50-50% 的叠加态。当你测量输出时，50% 的时间会是 `|0⟩`，50% 的时间会是 `|1⟩`。

概率约束：α² + β² = 1

那么，为什么 `α² + β²` 必须等于 1？为什么不能大于或小于 1？答案在于**概率**。总概率必须始终为 1。

- 如果 `α² + β²` 小于 1（例如 0.7），则意味着在 100 次测量中，你总共只会得到 `|0⟩` 或 `|1⟩` 70 次。
- 那么剩下的 30 次呢？输出必须是某种结果，而唯一可能的结果是 `|0⟩` 和 `|1⟩`。没有第三个选项。

因此，`α² + β²` 必须等于 1。这是验证一个量子态向量是否有效的基本检查。如果给你任何一个态向量，验证其有效性的一个简单方法就是检查 `α² + β²` 是否等于 1。如果不等于 1，那么它在量子计算中就是**一个无效的态向量**。

在经典机器上模拟量子态

像 `|0⟩` 这样的态向量是人类可以理解的。但是，如果你需要在传统机器上**模拟**量子电路，就必须以编程方式表示这些态向量。模拟意味着编写一个程序来建模电路从开始到结束的行为，包括每个门（如哈达玛门）的作用。

- 对量子模拟进行编程本质上是一场**向量操作的游戏**。
- 你从一个初始态向量开始（例如 `|0⟩`，表示为 `α=1, β=0`）。
- 然后你应用操作（门）来操作这个向量。例如，应用哈达玛门会改变这个态。
- 即使对于复杂的电路，其核心原理仍然是**态向量操作**。你从一个特定态开始，并随着时间的推移持续操作它，以获得最终结果。

State Vector Representation and Circuit Initialization

This is what the **state vector** is. After applying the **Hadamard gate**, you are in this new state. Even for complex circuits, the core principle remains **state vector manipulation**. You start from an initial state and continuously manipulate it over time to reach the final, designed state.

To simulate this manipulation, such as a Hadamard gate, you must write code. The first step is representing the state. This is done simply with an array.

- The input array corresponds to the initial state vector, e.g., `|0⟩` represented as `[1, 0]`.
- The final outcome, after the gate, might be `[1/√2, 1/√2]`.
- During programming, these state vectors are displayed as arrays. They can be row or column vectors; the key is using an array to represent the results.

Throughout the simulation, we rely on these state vector arrays to track the initial, intermediate, and output states.

Creating a Quantum Circuit Instance

We will now go back to the first simulation. As mentioned, this quantum circuit will be used to create an instance, which we then manipulate. We have already imported the necessary quantum circuit library.

Now, I am creating an instance of my circuit. Notice it takes two arguments:

1. The first argument specifies the **number of qubits** in the circuit.
2. The second argument specifies the **number of classical bits**.

Why are classical bits required? When a measurement occurs, the quantum state collapses. This collapsed, classical result needs to be stored somewhere—that's the purpose of the classical bits.

For example, in this circuit, there is one input qubit and one classical bit to store the measurement output. The configuration depends on your design:

- A circuit with 4 input qubits but only 2 needing measurement would be instantiated as `QuantumCircuit(4, 2)`.
- In general, if you have `n` qubits and need to measure `m` of them, you require `m` classical bits. Now, with the instance created, we can proceed.

状态向量表示与电路初始化

这就是**状态向量**。应用**哈达玛门**后，你就处于这个新状态。即使对于复杂电路，其核心原理仍然是**状态向量操作**。你从一个初始态开始，并随着时间的推移持续操作它，以达到最终设计的状态。

为了模拟这种操作，例如哈达玛门，你必须编写代码。第一步是表示状态。这可以简单地用一个数组来完成。

- 输入数组对应于初始状态向量，例如 `|0⟩` 表示为 `[1, 0]`。
- 经过门操作后的最终结果可能是 `[1/√2, 1/√2]`。
- 在编程过程中，这些状态向量以数组形式显示。它们可以是行向量或列向量；关键是用一个数组来表示结果。

在整个模拟过程中，我们依赖这些状态向量数组来跟踪初始态、中间态和输出态。

创建量子电路实例

我们现在回到第一个模拟。如前所述，这个量子电路将用于创建一个实例，然后我们对其进行操作。我们已经导入了必要的量子电路库。

现在，我正在创建我的电路的一个实例。注意它接受两个参数：

1. 第一个参数指定电路中的**量子比特数量**。
2. 第二个参数指定**经典比特的数量**。

为什么需要经典比特? 当进行测量时，量子态会坍缩。这个坍缩后的经典结果需要存储在某个地方——这就是经典比特的用途。

例如，在这个电路中，有一个输入量子比特和一个用于存储测量输出的经典比特。具体配置取决于你的设计：

- 一个有4个输入量子比特但只需要测量其中2个的电路，其实例化形式为 `QuantumCircuit(4, 2)`。
- 一般来说，如果你有 `n` 个量子比特并需要测量其中的 `m` 个，你就需要 `m` 个经典比特。现在，实例已经创建，我们可以继续了。

Quantum Circuit Instantiation and Measurement

Mohammad continues explaining the instantiation of a quantum circuit. Out of 4 input qubits, if you are only interested in measuring 2 qubits, you would instantiate it as `QuantumCircuit(4, 2)`. This principle applies generally: for `n` qubits where you need `m` measurements, you require `m` classical bits to store those results. The instance `QC1` is created to accommodate a single qubit and a single classical bit.

Clarifying Measurement and Classical Bits

A student interaction clarifies the relationship between qubits and classical bits during measurement.

> 🙋‍♂️ **Student**: So you just spoke about qubit and then classical qubit. 🤵 **Mohammad**: Yes, classical bit. Sorry, classical bit.
>
> 🙋‍♂️ **Student**: You measure only the classical bits. 🤵 **Mohammad**: We will measure the qubit but that measurement result gets stored in classical bits.
>
> 🙋‍♂️ **Student**: Yeah but how do you come to the conclusion that based on classical bits you measure and you will calculate? 🤵 **Mohammad**: No, no, we will measure. I will measure the qubit. I will not measure the classical bits. The measurement of the qubit gets stored in classical bits. Then obviously the classical bits retain that value. Then I will interpret my result based on that classical bits information.

The Role of Superposition and Collapse

The reason for this process is fundamental to quantum mechanics. Before measurement, qubits are in a state of **superposition**. After the measurement operation, the quantum state **collapses** to a definite classical value (0 or 1), which is then stored in the corresponding classical bit for further interpretation. This example makes the concept much clearer.

Step-by-Step Circuit Execution

Mohammad now walks through a very simple example line by line to demonstrate the process:

1. **Initialization**: A single qubit is initialized. Its default initial state is always `|0>`.
2. **Gate Application**: A **Hadamard gate** (`h`) is applied to the qubit in the next line of code.
3. **Measurement**: A measurement gate is applied. The syntax `measure(0, 0)` means: measure the 0th qubit and store the result in the 0th classical bit (indexing starts from 0 in computer science).
4. **Result Storage**: The measurement gate stores the collapsed result (0 or 1) into the specified classical register.

Simulation vs. Hardware Execution

After the circuit is designed, it needs to be simulated to see the results. The displayed circuit diagram is just a visual representation. The simulation step is necessary when you don't have physical quantum hardware. If you already had the hardware to run the circuit on, this simulation step would not be needed.

量子电路实例化与测量

Mohammad 继续解释量子电路的实例化。在4个输入量子比特中，如果只对测量其中2个感兴趣，则应将其实例化为 `QuantumCircuit(4, 2)`。这一原则普遍适用：对于 `n` 个量子比特，若需要进行 `m` 次测量，则需要 `m` 个经典比特来存储这些结果。实例 `QC1` 被创建，用于容纳一个量子比特和一个经典比特。

澄清测量与经典比特的关系

一段学生互动澄清了测量过程中量子比特与经典比特的关系。

> 🙋‍♂️ **学生**：所以你刚才提到了量子比特，然后是经典量子比特。 👨‍🏫 **Mohammad**：是的，经典比特。抱歉，是经典比特。
>
> 🙋‍♂️ **学生**：你只测量经典比特。 👨‍🏫 **Mohammad**：我们将测量量子比特，但测量结果会存储在经典比特中。
>
> 🙋‍♂️ **学生**：是的，但你是如何得出基于经典比特进行测量和计算的结论的？ 👨‍🏫 **Mohammad**：不，不，我们将进行测量。我将测量量子比特。我不会测量经典比特。量子比特的测量结果被存储在经典比特中。然后，经典比特显然会保留该值。之后，我将基于经典比特中的信息来解释我的结果。

叠加与坍缩的作用

这个过程的原因源于量子力学的基本原理。在测量之前，量子比特处于**叠加态**。测量操作之后，量子态**坍缩**为一个确定的经典值（0或1），然后该值被存储到相应的经典比特中，以供进一步解释。这个例子使概念更加清晰。

逐步电路执行

Mohammad 现在逐行演示一个非常简单的例子来说明该过程：

1. **初始化**：初始化一个量子比特。其默认初始状态始终是 `|0>`。
2. **门操作应用**：在下一行代码中，对量子比特应用一个**哈达玛门**（`h`）。

3. **测量**：应用一个测量门。语法 `measure(0, 0)` 表示：测量第0个量子比特，并将结果存储在第0个经典比特中（在计算机科学中，索引从0开始）。
4. **结果存储**：测量门将坍缩后的结果（0或1）存储到指定的经典寄存器中。

模拟与硬件执行

电路设计完成后，需要进行模拟以查看结果。显示的电路图只是一个可视化表示。当没有物理量子硬件时，模拟步骤是必要的。如果已经有可以运行该电路的硬件，则不需要此模拟步骤。

Simulating the Circuit

Now, I am just displaying and simulating the circuit. This is just a visual display of the circuit. Once my circuit is designed, I need to simulate it. If I had a hardware device, I would not need this simulation step.

If I already have a hardware device readily available, I could simply design this circuit and run it directly on the hardware. However, because I don't have hardware available right now, I need to rely on a simulator.

Using the Aer Simulator

For this purpose, I need to create an instance of a simulator. As I informed you earlier, I will use the **Aer simulator**. This is an Aer simulator instance. To this instance, I give a command to run, specifying which circuit it needs to execute—in this case, the `TC1` circuit—and how many times it needs to run.

- **Shots**: The number of times the circuit is executed. Here, it is set to 1024 (1k) shots.
- **Purpose**: I want the simulator to run it many times to verify my results and see if I am getting the general expected outcome.

Analyzing the Results

Once the job is done and run on the simulator, it stores the results. I need to analyze these results, specifically the count of how many times the outcome `0` arrived and how many times the outcome `1` arrived. This analysis involves getting the counts and plotting them in a histogram.

Expected Results: Hardware vs. Simulator

Now, what are you expecting? I have run this circuit 1024 times on the Aer simulator, which is a built-in simulator that comes freely with Qiskit.

Consider the expected result from two perspectives:

1. **On Ideal Hardware**: If you run this circuit on a fully reliable, tested, dedicated hardware device, what should you get? The expectation is a **50-50** distribution. It should count exactly 512 times for `0` and exactly 512 times for `1`. This is your real expectation when designing an algorithm for hardware to solve a problem.
2. **On a Simulator**: However, if you execute it on a simulator, it will not be exactly 512. There will be some errors or noise introduced into the circuit simulation.

Let's execute it and see what the result will be. This is my circuit.

模拟电路

现在，我只是在显示和模拟电路。这仅仅是电路的可视化显示。一旦我的电路设计完成，我就需要模拟它。如果我有一个硬件设备，我就不需要这个模拟步骤。

如果我已有现成的硬件设备，我可以直接设计这个电路并在硬件上运行。然而，因为我现在没有可用的硬件，所以我需要依赖模拟器。

使用 Aer 模拟器

为此，我需要创建一个模拟器实例。正如我之前告知你们的，我将使用 **Aer 模拟器**。这是一个 Aer 模拟器实例。我向这个实例发出运行命令，指定它需要执行哪个电路——本例中是 `TC1` 电路——以及需要运行的次数。

- **Shots（运行次数）**：电路被执行的次数。这里设置为 1024（1k）次。
- **目的**：我希望模拟器运行多次以验证我的结果，并查看是否得到了普遍预期的结果。

分析结果

一旦任务完成并在模拟器上运行，它会存储结果。我需要分析这些结果，特别是统计结果 `0` 出现了多少次，结果 `1` 出现了多少次。这个分析包括获取计数并将它们绘制在直方图中。

预期结果：硬件与模拟器

现在，你们期望什么？我已经在 Aer 模拟器上运行了这个电路 1024 次，Aer 模拟器是 Qiskit 免费内置的模拟器。

从两个角度考虑预期结果：

1. **在理想硬件上**：如果你在一个完全可靠、经过测试的专用硬件设备上运行这个电路，你应该得到什么？预期是 **50-50** 的分布。它应该精确地计数 512 次 `0` 和 512 次 `1`。这是你为硬件设计算法解决问题时的真实预期。
2. **在模拟器上**：然而，如果你在模拟器上执行，结果不会正好是 512。电路中会引入一些误差或噪声。

让我们执行它，看看结果会是什么。这就是我的电路。

Simulator Execution and Noise Introduction

When you execute the circuit on a simulator, the result will not be exactly 512 counts for `0` and `1`. Instead, some errors or **noise** are introduced into the circuit. For example, a measurement might yield 519 counts for `1` instead of the expected 512. However, if you increase the number of shots (e.g., to 10,000 or 20,000), the statistical gap between the simulated result and the ideal expectation becomes smaller. The simulator performs as expected but inherently includes some noise.

Noise in Quantum and Classical Systems

In quantum circuit design, the core circuit (e.g., applying a **Hadamard gate**) is correct. However, during execution—whether on a simulator or hardware—noise is inevitably embedded into the process. To address this, **noise cancellation circuits** can be designed. These are applied before measurement to filter out noise and achieve the desired, more accurate result.

This concept of error correction is not unique to quantum computing. Even in conventional digital computers, errors occur. For instance, when reading memory, a stored `0` might be flipped to a `1`. To correct this, circuits or software layers are embedded to detect and revert such unintended changes, employing a **filtering approach**.

The Path to Reliability

Conventional digital computers are highly reliable precisely because they have multiple layers of error correction and filtering. Quantum computing is approaching reliability in a similar way. A key component in achieving this is the design and integration of **noise**

cancellation circuits. Even with a perfectly designed quantum circuit, natural noise can be introduced during execution on any hardware, potentially spoiling the outcome. Therefore, incorporating noise mitigation strategies is essential.

模拟器执行与噪声引入

当你在模拟器上执行电路时，结果不会是精确的512次`0`和`1`计数。相反，电路中会引入一些误差或**噪声**。例如，测量结果可能显示`1`出现了519次，而不是预期的512次。然而，如果你增加执行的次数（例如，增加到10,000或20,000次），模拟结果与理想期望值之间的统计差距会变小。模拟器按预期运行，但本质上包含了一些噪声。

量子与经典系统中的噪声

在量子电路设计中，核心电路（例如，应用**Hadamard门**）是正确的。然而，在执行过程中——无论是在模拟器还是硬件上——噪声都不可避免地嵌入到过程中。为了解决这个问题，可以设计**噪声消除电路**。这些电路在测量之前应用，以过滤噪声并获得更准确、更符合期望的结果。

这种纠错概念并非量子计算独有。即使在传统的数字计算机中，也会发生错误。例如，在读取内存时，存储的`0`可能会翻转为`1`。为了纠正这种情况，系统中嵌入了电路或软件层来检测并恢复这种意外变化，采用了一种**过滤方法**。

实现可靠性的途径

传统的数字计算机之所以高度可靠，正是因为它们具有多层纠错和过滤机制。量子计算正以类似的方式接近可靠性。实现这一目标的一个关键部分是设计和集成**噪声消除电路**。即使量子电路设计完美，在任何硬件上执行时都可能引入自然噪声，从而可能破坏整个结果。因此，结合噪声缓解策略至关重要。

Noise Cancellation and Reliability

Even with a perfectly designed circuit, natural noise can be introduced when running it on any hardware, potentially spoiling the entire outcome. To recover the intended output, this noise must be canceled. A significant portion of quantum computing research is dedicated not only to problem-solving with **quantum algorithms** but also to developing **noise cancellation** and **noise filtering** surfaces. Achieving this standard reliability is crucial for people to trust and adopt the technology for their work.

Demonstrating Noise in Practice

When rerunning a quantum circuit, the results are not always identical. For instance, one run might yield counts of 510 and 514, while another yields 545 and 479. However, the results are typically near each other. Observing the histogram reveals that the gap between different outcome counts is relatively small. If interested, the specific results can be shared for analysis.

Practical Exercise: Circuit Analysis

Now, answer these questions after running the cell above.

Question 1: Why do we not get only 0 as the result?

> 👩‍🏫 Lecturer: Because the **Hadamard gate** is supposed to create superposition. We have applied the Hadamard gate, and it should give both 0 and 1 in a state of superposition.

Question 2: Are the counts exactly equal? Why or why not?

> 👩‍🏫 Lecturer: The counts are not exactly equal. The reason is that noise is present, and we currently lack a circuit to filter out that noise, as we are relying solely on a simulator.

Question 3: What will happen if the Hadamard gate is removed?

> 👩‍🏫 Lecturer: Before finding the result, visualize it. What will be the result? If you comment out the Hadamard gate line in the code, what will happen? 🙋‍♂️ Student: It can be anything. 👩‍🏫 Lecturer: Why can it be anything? 🙋‍♂️ Student: It will be zero. 👩‍🏫 Lecturer: It will be zero. Because you are initializing the qubit to |0〉, you are not applying any gate to change its state, and you are just measuring it. So it should be zero. Please do it: comment out the line `qc.h(0)` and give me the result, confirming whether it will be zero or not.

噪声消除与可靠性

即使电路设计完美，在任何硬件上运行时都可能引入自然噪声，从而可能破坏整个结果。为了恢复预期的输出，必须消除这种噪声。量子计算研究的很大一部分不仅致力于使用**量子算法**解决问题，还致力于开发**噪声消除**和**噪声过滤**表面。实现这种标准可靠性对于人们信任并采用该技术进行工作至关重要。

实践中的噪声演示

重新运行量子电路时，结果并不总是相同的。例如，一次运行可能产生计数510和514，而另一次运行产生545和479。然而，结果通常彼此接近。观察直方图可以发现，不同结果计数之间的差距相对较小。如果有兴趣，可以分享具体结果进行分析。

实践练习：电路分析

现在，运行完上面的单元格后回答这些问题。

问题1：为什么我们得到的结果不只是0?

> 👩‍🏫 讲师：因为**哈达玛门**旨在创建叠加态。我们应用了哈达玛门，它应该以叠加态的形式同时给出0和1。

问题2：计数完全相等吗？为什么或为什么不?

> 👩‍🏫 讲师：计数并不完全相等。原因是存在噪声，并且我们目前缺乏过滤该噪声的电路，因为我们仅仅依赖于模拟器。

问题3：如果移除哈达玛门会发生什么?

> 👩‍🏫 讲师：在找出结果之前，先将其可视化。结果会是什么？如果你在代码中注释掉哈达玛门那一行，会发生什么？ 🙋‍♂️ 学生：可能是任何结果。 👩‍🏫 讲师：为什么可能是任何结果？ 🙋‍♂️ 学生：会是零。 👩‍🏫 讲师：会是零。因为你将量子比特初始化为|0⟩，你没有应用任何门来改变其状态，你只是测量它。所以它应该是零。请这样做：注释掉 `qc.h(0)` 这一行，然后给我结果，确认它是否为零。

Practical Exercise: Single Qubit State Verification

You should get a result of `0`. Please apply this change and run the circuit.

Three-Step Verification Exercise

To build confidence and understanding, please complete these three steps:

1. **Modify the circuit**: Remove the **Hadamard gate** and run the simulation again. You will always get a result of `0`, regardless of the number of measurement shots (e.g., 1024).

2. **Replace the gate**: Substitute the Hadamard gate with an **X gate**.
3. **Observe the result**: Run the circuit. The result will always be `1`, no matter how many times you measure it.

After completing these steps, we will proceed to the topic of **two-qubit superposition**.

Understanding the State Vector

Initial State and Notation

What is the **state vector**? In mathematical terms, how do you write it as an array?

- The initial state vector is `|0⟩`. This is the **Dirac (bra-ket) notation** used in quantum physics to represent quantum states. `|0⟩` represents the zero state.
- For simulation purposes, you represent it differently. The state `|0⟩` is represented by the column vector `[1, 0]`.

State Vector Representation and Function

To provide more context on how the **state vector** functions:

- It is fundamentally a **column vector**. We often write it in a row for convenience, but consider it as a column: `[1, 0]^T`.
- This vector has two positions (indices):
  - **Index 0** corresponds to the basis state `|0⟩`.
  - **Index 1** corresponds to the basis state `|1⟩`.
- The value at each index represents the **probability amplitude** for that basis state.
  - Currently, for `|0⟩`, the amplitude is `1`.
  - For `|1⟩`, the amplitude is `0`.

After applying the **X gate**, the state becomes `|1⟩`. This translates to the state vector `[0, 1]`, meaning the amplitude moves from index 0 to index 1. This is the essence of what the state vector represents.

I use the array notation `[1, 0]` because it is more concise for writing and simulation than always using the full Dirac notation.

实践练习：单量子比特状态验证

你应该得到结果为 `0`。请应用此更改并运行电路。

三步验证练习

为了建立信心和理解，请完成以下三个步骤：

1. **修改电路**：移除 **Hadamard 门** 并再次运行模拟。无论测量次数多少（例如 1024 次），你将始终得到结果 `0`。
2. **替换门操作**：将 Hadamard 门替换为 **X 门**。
3. **观察结果**：运行电路。结果将始终为 `1`，无论你测量多少次。

完成这些步骤后，我们将继续学习 **双量子比特叠加** 的主题。

理解状态向量

初始状态与符号表示

什么是 **状态向量**？用数学术语来说，如何将其写成一个数组？

- 初始状态向量是 `|0⟩`。这是量子物理学中用来表示量子态的 **狄拉克（bra-ket）符号**。`|0⟩` 表示零状态。
- 出于模拟目的，你需要用不同的方式表示它。状态 `|0⟩` 由列向量 `[1, 0]` 表示。

状态向量的表示与功能

为了更详细地说明 **状态向量** 的功能：

- 它本质上是一个 **列向量**。我们通常为了方便写成一行，但请将其视为一列：`[1, 0]^T`。
- 这个向量有两个位置（索引）：
  - **索引 0** 对应基态 `|0⟩`。
  - **索引 1** 对应基态 `|1⟩`。
- 每个索引处的值代表该基态的 **概率幅**。
  - 目前，对于 `|0⟩`，其幅值为 `1`。
  - 对于 `|1⟩`，其幅值为 `0`。

应用 **X 门** 后，状态变为 `|1⟩`。这转换为状态向量 `[0, 1]`，意味着幅值从索引 0 移动到了索引 1。这就是状态向量所代表的本质。

我使用数组符号 `[1, 0]` 是因为与始终使用完整的狄拉克符号相比，它在书写和模拟中更为简洁。

Notation and State Vector Representation

The value you write corresponds to the state `|0⟩`. Inside, you write the **magnitude** (or probability amplitude). Currently, the magnitudes are `1` and `0`. Applying an **X gate** translates the amplitude from index 0 to index 1. This is the essence of what the state vector represents.

I use the array notation `[1, 0]` because it takes just one line of writing, whereas the full Dirac notation would take two lines. This is simply for conciseness in writing and simulation. During programming, you can represent it in any way, but this particular notation makes the most logical sense.

Lab Work Follow-up

Now, let's address the lab exercise. Have you all completed it? It shouldn't take more than 2-3 minutes. You were supposed to do two things:

1. Remove the **H gate** and observe the result.
2. Remove the **H gate**, then apply an **X gate**, and observe the result.

If you haven't done it yet, please take some time after class to complete it. You must confirm you have done all tasks in the notebook before joining the next class.

Two-Qubit Superposition

We now move to **two-qubit superposition**. The problem is: we have two qubits and need to put both into superposition. We have already seen single-qubit superposition.

- How many states will this demonstrate? We have two qubits, q1 and q2.
- Which gate is used for superposition? The **Hadamard gate**.
- What is the initial state for each qubit? `|0⟩`.
- What is the final state for a single qubit after a Hadamard? `1/√2 (|0⟩ + |1⟩)`.

Therefore, for each qubit individually, the final state is `1/√2 (|0⟩ + |1⟩)`.

System View and Initial State Vector

We need to read this as a **consolidated circuit** or a single **system** composed of two subsystems (the two qubits). We must understand it as a whole.

For this two-qubit system, what is the **initial state**? It is `|00⟩`. In Dirac notation, a quantum physicist would write this as `|0⟩ ⊗ |0⟩` or simply `|00⟩`.

> 👩‍🏫 Lecturer: What is my initial state vector now? 👨‍🎓 Student: 0, 0.

However, as a programmer implementing this, you need a different notation for computation—like a **state vector notation**.

完整中文翻译

符号与状态向量表示

你写的值对应状态 `|0⟩`。在里面，你写入的是**幅度**（或称概率幅）。当前的幅度是 `1` 和 `0`。应用 **X 门** 会将幅度从索引 0 转换到索引 1。这就是状态向量所代表的本质。

我使用数组符号 `[1, 0]` 是因为它只需一行书写，而完整的狄拉克符号则需要两行。这只是为了书写和模拟的简洁性。在编程时，你可以用任何方式表示它，但这种特定的符号在逻辑上最有意义。

实验工作跟进

现在，我们来处理实验练习。你们都完成了吗？这应该不需要超过 2-3 分钟。你们需要做两件事：

1. 移除 **H 门** 并观察结果。
2. 移除 **H 门**，然后应用 **X 门**，并观察结果。

如果还没有完成，请在课后花些时间完成。在参加下一节课之前，你必须确认已完成笔记本中的所有任务。

双量子比特叠加

我们现在进入**双量子比特叠加**。问题是：我们有两个量子比特，需要将两者都置于叠加态。我们已经见过单量子比特叠加。

- 这将展示多少种状态？我们有两个量子比特，q1 和 q2。
- 用于叠加的门是哪个？**哈达玛门**。
- 每个量子比特的初始状态是什么？`|0⟩`。
- 单个量子比特经过哈达玛门后的最终状态是什么？`1/√2 (|0⟩ + |1⟩)`。

因此，对于每个独立的量子比特，其最终状态是 `1/√2 (|0⟩ + |1⟩)`。

系统视图与初始状态向量

我们需要将其视为一个**整合的电路**或一个由两个子系统（两个量子比特）组成的单一**系统**。我们必须将其作为一个整体来理解。

对于这个双量子比特系统，**初始状态**是什么？是 `|00⟩`。在狄拉克符号中，量子物理学家会将其写为 `|0⟩ ⊗ |0⟩` 或简写为 `|00⟩`。

> 👩‍🏫 讲师：我现在的初始状态向量是什么？ 🙋 学生：0, 0。

然而，作为实现此功能的程序员，你需要一种不同的符号进行计算——比如**状态向量符号**。

> 👩‍🏫 Lecturer: What is my initial state vector now? 🙋 Student: 0, 0.

For a quantum physicist using Dirac notation, `|00⟩` is sufficient. However, as a programmer implementing this, you need a different notation, like a **state vector notation** or an array notation.

State Vector Representation for Two Qubits

The initial state is `|00⟩`. To represent this as a state vector (an array), we must determine its size. For a system of two qubits, the state vector has `4 rows` and `1 column` (a `4x1` column vector). This is because the number of basis states is `2^n`, where `n=2` (qubits).

The basis states and their corresponding positions in the vector are:

- `|00⟩` corresponds to the first position (index 0).
- `|01⟩` corresponds to the second position.
- `|10⟩` corresponds to the third position.
- `|11⟩` corresponds to the fourth position.

Therefore, the **initial state vector** for `|00⟩` is:

```
[1, 0, 0, 0]^T
```

This is a `4x1` column vector. The `1` is the **amplitude** for the state `|00⟩`. The square of the amplitude gives the probability. Here, `1^2 = 1`, meaning there is a 100% probability of measuring the system in the state `|00⟩`.

Interpreting the Final State Vector

After applying operations (like a Hadamard gate to each qubit), the final state vector will also be `4x1`. Its entries will be the amplitudes for each possible two-qubit basis state.

> 👨‍🏫 Lecturer: When you measure it, what are the expected outcomes? 🙋 Student: 50, 50.

For the final state (e.g., after applying H to both qubits), each individual qubit has a 50% chance of being `0` or `1`. However, we must consider the **system** as a whole. The possible measurement outcomes for the two-qubit system are the four basis states: `|00⟩`, `|01⟩`, `|10⟩`, and `|11⟩`.

完整中文翻译

> 👨‍🏫 讲师：我现在的初始状态向量是什么？ 🙋 学生：0, 0。

对于使用狄拉克符号的量子物理学家来说，`|00⟩` 就足够了。然而，作为实现此功能的程序员，你需要一种不同的符号，比如**状态向量符号**或数组符号。

双量子比特的状态向量表示

初始状态是 `|00⟩`。要将其表示为状态向量（一个数组），我们必须确定其大小。对于一个双量子比特系统，状态向量有 `4 行` 和 `1 列`（一个 `4x1` 列向量）。这是因为基态的数量是 `2^n`，其中 `n=2`（量子比特数）。

基态及其在向量中的对应位置是：

- `|00⟩` 对应第一个位置（索引 0）。
- `|01⟩` 对应第二个位置。

- `|10⟩` 对应第三个位置。
- `|11⟩` 对应第四个位置。

因此，`|00⟩` 的**初始状态向量**是：

```
[1, 0, 0, 0]^T
```

这是一个 `4x1` 列向量。其中的 `1` 是状态 `|00⟩` 的**振幅**。振幅的平方给出概率。这里，`1^2 = 1`，意味着测量系统处于 `|00⟩` 状态的概率是 100%。

解释最终状态向量

在应用操作（例如对每个量子比特应用哈达玛门）之后，最终状态向量也将是 `4x1`。其条目将是每个可能的双量子比特基态的振幅。

> 👩‍🏫 讲师：当你测量它时，预期的结果是什么？ 🙋‍♂️ 学生：50, 50。

对于最终状态（例如，对两个量子比特都应用 H 门后），每个单独的量子比特有 50% 的几率是 `0` 或 `1`。然而，我们必须将整个**系统**作为一个整体来考虑。双量子比特系统可能的测量结果是四个基态：`|00⟩`、`|01⟩`、`|10⟩` 和 `|11⟩`。

Two-Qubit System State Vector and Measurement

System Combinations and Probabilities

As a system, the two qubits can have several possible combinations. The first qubit can be `0` and the second qubit can be `0` simultaneously, giving `|00⟩`. Similarly, the other possible combinations are `|01⟩`, `|10⟩`, and `|11⟩`.

Now, we must determine the probabilities of these possibilities. Since each individual qubit is in an **equal superposition** (50% `0`, 50% `1`), all four system combinations are equally likely. With four equally possible combinations, the individual probability for each is 0.25, or 25%.

- The probability of `|00⟩` is 25%.
- The probability of `|01⟩` is 25%.
- The probability of `|10⟩` is 25%.

- The probability of `|11⟩` is 25%.

State Vector Representation

The **amplitude** for each basis state is the square root of its probability. Since the probability is 1/4, the amplitude is 1/2. The complete **output state vector** can be written as:

```
( |00⟩ + |01⟩ + |10⟩ + |11⟩ ) / 2
```

This means the amplitudes are `[1/2, 1/2, 1/2, 1/2]`. This state is a **superposition** of the individual basis states `|00⟩`, `|01⟩`, `|10⟩`, and `|11⟩`. When you measure it, any of these four results can appear, each with a 25% probability.

Quantum Circuit Implementation

The circuit for this experiment has 2 qubits and 2 classical bits. The operations are applied sequentially:

1. Apply the **Hadamard (H) gate** on qubit at index 0 (the first qubit).
2. Apply the **Hadamard (H) gate** on qubit at index 1 (the second qubit).
3. Measure qubit 0 into classical bit 0.
4. Measure qubit 1 into classical bit 1.

We then initialize a simulator, run the circuit (e.g., for 1024 shots), get the result counts, and display them. The code structure remains largely the same, with only the circuit definition changed.

Expected Measurement Results

The circuit diagram shows qubits Q0 and Q1, and two classical bits. With 1024 shots, the expected count for each of the four states (`00`, `01`, `10`, `11`) is 25% of 1024, which is 256.

双量子比特系统状态向量与测量

系统组合与概率

作为一个系统，两个量子比特可以有几种可能的组合。第一个量子比特可以是 `0`，同时第二个量子比特也可以是 `0`，得到 `|00⟩`。类似地，其他可能的组合是 `|01⟩`、`|10⟩` 和 `|11⟩`。

现在，我们必须确定这些可能性的概率。由于每个单独的量子比特都处于**均匀叠加态**（50% 为 `0`，50% 为 `1`），所有四个系统组合的可能性均等。对于四个可能性均等的组合，每个组合的单独概率是 0.25，即 25%。

- `|00⟩` 的概率是 25%。
- `|01⟩` 的概率是 25%。
- `|10⟩` 的概率是 25%。
- `|11⟩` 的概率是 25%。

状态向量表示

每个基态的**振幅**是其概率的平方根。由于概率是 1/4，振幅就是 1/2。完整的**输出状态向量**可以写成：

```
( |00⟩ + |01⟩ + |10⟩ + |11⟩ ) / 2
```

这意味着振幅是 `[1/2, 1/2, 1/2, 1/2]`。这个状态是基态 `|00⟩`、`|01⟩`、`|10⟩` 和 `|11⟩` 的**叠加**。当你测量它时，这四个结果中的任何一个都可能出现，每个出现的概率为 25%。

量子电路实现

这个实验的电路有 2 个量子比特和 2 个经典比特。操作按顺序应用：

1. 在索引 0 的量子比特（第一个量子比特）上应用**哈达玛（H）门**。
2. 在索引 1 的量子比特（第二个量子比特）上应用**哈达玛（H）门**。
3. 将量子比特 0 测量到经典比特 0 中。
4. 将量子比特 1 测量到经典比特 1 中。

然后我们初始化一个模拟器，运行电路（例如，运行 1024 次），获取结果计数并显示它们。代码结构基本保持不变，只改变了电路定义。

预期测量结果

电路图显示了量子比特 Q0 和 Q1，以及两个经典比特。运行 1024 次，四个状态（`00`、`01`、`10`、`11`）中每一个的预期计数是 1024 的 25%，即 256。

Circuit Execution Results and Modifying Input

Observed Measurement Counts

The circuit has been executed. It involves two quantum bits (Q0, Q1) and two classical bits for measurement.

- The first quantum bit (0th index) is measured into the first classical bit.
- The second quantum bit (1st index) is measured into the second classical bit.
- The circuit was run for 1024 shots.

The expected count for each of the four possible states (`00` , `01` , `10` , `11`) is 25% of 1024, which is 256. The observed counts are very close to this expectation: 266, 268, 268, and 267.

Interactive Exercise: Applying an X Gate Before Hadamard

The lecturer then poses an exercise to modify the circuit and observe the change in results.

> 👩‍🏫 Lecturer: Now, apply an **X gate** to qubit 0 *before* applying the **Hadamard gate**. How will the results change? Will there be any impact? Make this change. The point is, you are now giving an input of `|1⟩` to the Hadamard gate. So, if you give `|1⟩` to a Hadamard gate, what will be the result?

All students are instructed to perform this modification in their code.

Conceptual Review: Hadamard Gate on |1⟩

The lecturer reviews the expected outcome from a conceptual standpoint, referencing previous work with a single qubit.

- Applying an **X gate** to `|0⟩` flips it to `|1⟩` .
- Feeding `|1⟩` into a **Hadamard gate** creates a superposition.
- The output state vector in this case is `(|0⟩ − |1⟩) / √2` .

This is different from the superposition created by applying Hadamard to `|0⟩`, which yields `(|0⟩ + |1⟩) / √2`. Both result in an **equal superposition** where measurement yields `0` or `1` with 50% probability each. The difference lies in the relative phase (the minus sign) between the `|0⟩` and `|1⟩` components in the state vector, which is not directly observable from simple measurement counts alone.

电路执行结果与修改输入

观测到的测量计数

电路已执行完毕。它涉及两个量子比特（Q0, Q1）和两个用于测量的经典比特。

- 第一个量子比特（第 0 个索引）被测量到第一个经典比特中。
- 第二个量子比特（第 1 个索引）被测量到第二个经典比特中。
- 电路运行了 1024 次。

四个可能状态（`00`、`01`、`10`、`11`）中每一个的预期计数是 1024 的 25%，即 256。观测到的计数非常接近这个预期值：266、268、268 和 267。

互动练习：在哈达玛门之前应用 X 门

讲师随后提出一个练习：修改电路并观察结果的变化。

> 讲师：现在，在应用**哈达玛门** *之前*，对量子比特 0 应用一个 **X 门**。结果会如何改变？会有任何影响吗？进行这个修改。关键在于，你现在是给哈达玛门输入 `|1⟩`。那么，如果你给哈达玛门输入 `|1⟩`，结果会是什么？

所有学生都被指示在他们的代码中执行此修改。

概念回顾：对 |1⟩ 应用哈达玛门

讲师从概念角度回顾了预期结果，并参考了之前对单个量子比特的工作。

- 对 `|0⟩` 应用 **X 门** 会将其翻转为 `|1⟩`。
- 将 `|1⟩` 输入**哈达玛门**会创建一个叠加态。
- 这种情况下的输出状态向量是 `(|0⟩ - |1⟩) / √2`。

这与对 `|0⟩` 应用哈达玛门产生的叠加态不同，后者产生 `(|0⟩ + |1⟩) / √2`。两者都会导致一个**相等的叠加态**，测量时各有 50% 的概率得到 `0` 或 `1`。区别在于状态向量中 `|0⟩` 和 `|1⟩` 分量之间的相对相位（负号），仅从简单的测量计数中无法直接观察到这一点。

H3: Distinguishing Two Superposition States

In terms of normal understanding, both `|0⟩` and `|1⟩` will appear when measured. When you measure, sometimes `0` will appear, sometimes `1` will appear with equal proportion. There is an **equal superpositioning**, but that `|1⟩` is different.

The state vector is `(|0⟩ − |1⟩) / √2`. If we write this state vector, it is `(1/√2)|0⟩ + (−1/√2)|1⟩`. That means it's a combination of `|0⟩` and `|1⟩`, but with these specific amplitudes. One amplitude is positive (`1/√2`) and the other is negative (`−1/√2`).

The key point is: what does this `−1/√2` mean? What will it do? In terms of waves, this minus sign indicates a **phase difference**. This minus sign is introducing a **phase difference of 180 degrees**. This `|1⟩` is not exactly the same as the one from before.

H3: Comparing Phase Relationships

This state gives you the same `1/√2` amplitude for `|1⟩`, but it is in a different direction altogether due to the phase difference. When we exclusively discuss phase difference later, this will make more sense.

- The state from applying a **Hadamard gate** to `|0⟩` is `(|0⟩ + |1⟩) / √2`. Both amplitudes are positive, meaning `|0⟩` and `|1⟩` are **in phase**.
- The state here is `(|0⟩ − |1⟩) / √2`. The amplitude for `|1⟩` is negative, introducing a **phase difference**. This means `|0⟩` and `|1⟩` are not synced; they are poles apart.

Earlier, `|0⟩` and `|1⟩` were synced in the same direction. Now, `|1⟩` is in a different direction altogether. This `|1⟩` is not exactly the same, though when you interpret the measurement result, both `0` and `1` will appear with equal probability. The superposition is still there, but it's a different kind of superposition.

H3: Observability of the Phase

In terms of noticing the result, when you run the circuit, `00` and `01` will come up. You will not notice any changes in the measurement counts. Unless you output the **state vector** itself, the state vector will reveal this minus sign (`−1/√2`).

> 👩‍🏫 Lecturer: Maybe in today's class or next class, I will show you how to see the state vector as well. Now, I don't have enough time, but I want you to visualize.

H3: Thought Experiment: Applying X vs. Z Gates

Consider the same 2-qubit circuit. Apply an **X gate** on qubit 0 before measurement. Apply a **Z gate** on qubit 0 before measurement. What will be the differences in both these two situations?

- Both `X` and `Z` are like a NOT gate in a sense.
- However, there is a slight difference, and the slight difference is only in terms of **phase introduction**.
- When you apply a **Z gate**, it will also introduce a phase shift.

完整中文翻译

H3: 区分两种叠加态

按照通常的理解，测量时 `|0⟩` 和 `|1⟩` 都会出现。当你测量时，有时会出现 `0`，有时会出现 `1`，且比例相等。存在一个**相等的叠加态**，但这里的 `|1⟩` 是不同的。

其状态向量是 `(|0⟩ - |1⟩) / √2`。如果我们写出这个状态向量，它是 `(1/√2)|0⟩ + (-1/√2)|1⟩`。这意味着它是 `|0⟩` 和 `|1⟩` 的组合，但具有这些特定的振幅。一个振幅是正的 (`1/√2`)，另一个是负的 (`-1/√2`)。

关键点是：这个 `-1/√2` 意味着什么？它会做什么？从波的角度来看，这个负号表示**相位差**。这个负号引入了**180度的相位差**。这个 `|1⟩` 与之前得到的 `|1⟩` 并不完全相同。

H3: 比较相位关系

这个状态给你相同的 `|1⟩` 振幅 `1/√2`，但由于相位差，它完全处于不同的方向。当我们后面专门讨论相位差时，这会更容易理解。

- 对 `|0⟩` 应用**哈达玛门**得到的状态是 `(|0⟩ + |1⟩) / √2`。两个振幅都是正的，意味着 `|0⟩` 和 `|1⟩` **同相**。
- 这里的状态是 `(|0⟩ - |1⟩) / √2`。`|1⟩` 的振幅是负的，引入了**相位差**。这意味着 `|0⟩` 和 `|1⟩` 不同步；它们截然相反。

之前，`|0⟩` 和 `|1⟩` 在同一方向上同步。现在，`|1⟩` 完全处于不同的方向。这个 `|1⟩` 并不完全相同，尽管当你解释测量结果时，`0` 和 `1` 都会以相等的概率出现。叠加态仍然存在，但它是另一种叠加态。

H3: 相位的可观测性

从观察结果的角度来看，当你运行电路时，会出现 `00` 和 `01`。你在测量计数中不会注意到任何变化。除非你输出**状态向量**本身，否则状态向量中的这个负号 (`-1/√2`) 是无法直接观察到的。

> 🤷 讲师：也许在今天或下节课，我也会向你们展示如何查看状态向量。现在，我没有足够的时间，但我希望你们能想象一下。

H3: 思考实验：应用 X 门与 Z 门

考虑相同的 2 量子比特电路。在测量前对量子比特 0 应用一个 **X 门**。在测量前对量子比特 0 应用一个 **Z 门**。这两种情况下的区别是什么?

- 从某种意义上说，`X` 和 `Z` 都类似于非门。
- 然而，存在一个细微的差别，而这个细微的差别仅在于**相位的引入**。
- 当你应用 **Z 门**时，它也会引入一个相移。

H3: 思考实验：应用 X 门与 Z 门的区别

那么，在这两种情况下会有什么不同呢?

- 如果你在量子比特0上应用 **X 门**，它和 **Z 门** 一样，都是一种非门。
- 但我之前提到过，它们之间存在一个细微的差别，这个差别仅在于**相位的引入**。
- 当你应用 **Z 门** 时，它也会将 |0⟩ 翻转为 |1⟩，但同时会引入一个相位差。
- 因此，从**解释测量结果**的角度看，两者都将0翻转为1，结果没有变化。
- 但从**状态向量**的角度看，两者存在巨大的差异。

由于我们尚未深入讨论 Z 门和 Y 门，且这涉及更深层的概念，今天不适合深入探讨。我们暂时点到为止。

H3: 课程安排与中场休息

现在时间是 12:04。我们将休息 10 分钟，于 12:15 回来，并完成**纠缠**部分的内容。

H3: 抽象层面的概念：哈达玛门

到目前为止，我们讨论了**叠加**，以及如何通过**哈达玛门**实现它。

- 即使在概念层面，哈达玛门也是存在的。
- 想象一下，即使目前没有能实现它的硬件，但在抽象概念上，哈达玛门是存在的。
- 如果我们输入 |0⟩ 到哈达玛门，理论上它将输出一个叠加态。
- 未来，如果有硬件能实现哈达玛门的功能，那么在概念和实现上，我们就完成了目标。

> 👩‍🏫 Lecturer: 明白我的意思吗?

我们是在抽象层面学习：存在一个哈达玛门，在模拟中它的作用就是产生叠加态。至于它如何具体实现，那是量子硬件工程师的工作。技术正在发展以使其成为可能，但在概念层面，我们需要在抽象层次上工作。

我们知道有一个哈达玛门，并且需要用它来设计电路。我们应该知道它的功能：它将使量子比特进入叠加态。

H3: 下一个概念：量子纠缠

接下来要讨论的概念是**纠缠**。我们上节课讨论过纠缠是什么?

- 纠缠是指两个事物变得**相互关联**。
- 我们必须以某种方式使两个事物相关联，以至于其中一个发生的变化会自动影响另一个。
- 为此，我们需要设计一个方案（或电路），这是我们当前要解决的问题。

同样，在概念层面，存在一些特定的门（如 **CNOT 门**）可以实现这一点。我们将应用并证明这类关联是可以存在的。

- **CNOT 门**将扮演重要角色，我们稍后会讨论它。
- 但大家都了解 **NOT 门**吗？NOT 门是什么?

完整中文翻译

H3: 思考实验：应用 X 门与 Z 门的区别

那么，在这两种情况下会有什么不同呢?

- 如果你在量子比特0上应用 **X 门**，它和 **Z 门** 一样，都是一种非门。
- 但我之前提到过，它们之间存在一个细微的差别，这个差别仅在于**相位的引入**。
- 当你应用 **Z 门** 时，它也会将 |0〉翻转为 |1〉，但同时会引入一个相位差。
- 因此，从**解释测量结果**的角度看，两者都将0翻转为1，结果没有变化。
- 但从**状态向量**的角度看，两者存在巨大的差异。

由于我们尚未深入讨论 Z 门和 Y 门，且这涉及更深层的概念，今天不适合深入探讨。我们暂时点到为止。

H3: 课程安排与中场休息

现在时间是 12:04。我们将休息 10 分钟，于 12:15 回来，并完成**纠缠**部分的内容。

H3: 抽象层面的概念：哈达玛门

到目前为止，我们讨论了**叠加**，以及如何通过**哈达玛门**实现它。

- 即使在概念层面，哈达玛门也是存在的。
- 想象一下，即使目前没有能实现它的硬件，但在抽象概念上，哈达玛门是存在的。
- 如果我们输入 |0〉到哈达玛门，理论上它将输出一个叠加态。
- 未来，如果有硬件能实现哈达玛门的功能，那么在概念和实现上，我们就完成了目标。

> 👨‍🏫 Lecturer: 明白我的意思吗?

我们是在抽象层面学习：存在一个哈达玛门，在模拟中它的作用就是产生叠加态。至于它如何具体实现，那是量子硬件工程师的工作。技术正在发展以使其成为可能，但在概念层面，我们需要在抽象层次上工作。

我们知道有一个哈达玛门，并且需要用它来设计电路。我们应该知道它的功能：它将使量子比特进入叠加态。

H3: 下一个概念：量子纠缠

接下来要讨论的概念是**纠缠**。我们上节课讨论过纠缠是什么？

- 纠缠是指两个事物变得**相互关联**。
- 我们必须以某种方式使两个事物相关联，以至于其中一个发生的变化会自动影响另一个。
- 为此，我们需要设计一个方案（或电路），这是我们当前要解决的问题。

同样，在概念层面，存在一些特定的门（如 **CNOT 门**）可以实现这一点。我们将应用并证明这类关联是可以存在的。

- **CNOT 门**将扮演重要角色，我们稍后会讨论它。
- 但大家都了解 **NOT 门**吗？NOT 门是什么？

Quantum Computing: Conditional NOT Gate and Circuit Analysis

The Conditional NOT (CNOT) Gate

This is our current problem. Conceptually, there exists a specific type of gate. We will apply and prove that this kind of correlation can exist.

- The **CNOT gate** will play an important role.
- Everyone knows the **NOT gate**? What is a NOT gate? It maps `0 -> 1` and `1 -> 0`.

We now want to make a NOT gate functional, but *conditionally*. There will be an additional conditional input line to it. Therefore, a **CNOT gate** has two inputs.

- One input is the normal target qubit that needs to be flipped.
- The other input is a conditional control qubit.

The gate operates as follows:

1. When the conditional control qubit is `1`, the NOT gate is functional and flips the target qubit.
2. When the conditional control qubit is not `1` (i.e., it is `0`), the gate is dysfunctional. It does not operate; it simply passes the target input to the output without flipping it.

That is what the **Conditional NOT (CNOT) gate** is, and we are going to use it.

Analyzing a Two-Qubit Circuit

I will explain it in detail later, but first, let's examine the circuit. This is my circuit right now:

- You have two qubits, `A` and `B`.
- First, I apply the **Hadamard gate** to one of them.
- These are my two input lines: one is the conditional control line, and the other is the normal target input line for the **CNOT gate**.
- This is the standard notation for a CNOT gate. This is my simple circuit.
- Finally, I measure both qubits.

Expected Output Discussion

What output are you expecting? This is a simple circuit with two qubits, a Hadamard gate, and a CNOT gate. On an abstract level, this is what the Hadamard gate does, and this is what the CNOT gate does.

If you design a circuit like this, you will get something amazing. What are we expecting on the output lines? We have two qubits. How many output combinations are possible? **Four**.

In our last circuit, we saw that with two qubits and a Hadamard gate, you exhaust all four possibilities. You get all four possible states. Here, we have done a simple twist: we applied the Hadamard gate, and instead of applying another Hadamard to the next qubit, we applied a CNOT gate.

量子计算：条件非门与电路分析

条件非门 (CNOT Gate)

这是我们当前要解决的问题。从概念上讲，存在一种特定类型的门。我们将应用并证明这种关联是可以存在的。

- **CNOT 门**将扮演重要角色。
- 大家都了解**非门 (NOT gate)**吗？非门是什么？它将 `0 -> 1` 和 `1 -> 0`。

我们现在想让一个非门功能化，但是*有条件地*。它将有一个额外的条件输入线。因此，一个**CNOT 门**有两个输入。

- 一个输入是需要被翻转的常规目标量子比特。
- 另一个输入是条件控制量子比特。

该门的工作原理如下：

1. 当条件控制量子比特为 `1` 时，非门功能化并翻转目标量子比特。
2. 当条件控制量子比特不为 `1`（即它为 `0`）时，该门不工作。它不操作；只是将目标输入原样传递到输出，而不进行翻转。

这就是**条件非门 (CNOT Gate)**，我们将要使用它。

分析一个双量子比特电路

我稍后会详细解释，但首先，让我们检查这个电路。这是我当前的电路：

- 你有两个量子比特，`A` 和 `B`。
- 首先，我对其中一个应用**哈达玛门 (Hadamard gate)**。
- 这是我的两条输入线：一条是条件控制线，另一条是**CNOT 门**的常规目标输入线。
- 这是 CNOT 门的标准符号。这是我的简单电路。
- 最后，我测量两个量子比特。

预期输出讨论

你期望什么输出？这是一个包含两个量子比特、一个哈达玛门和一个 CNOT 门的简单电路。在抽象层面上，这是哈达玛门的作用，也是 CNOT 门的作用。

如果你这样设计一个电路，你会得到一些令人惊奇的结果。我们在输出线上期望什么？我们有两个量子比特。有多少种可能的输出组合？**四种**。

在我们上一个电路中，我们看到，对于两个量子比特和一个哈达玛门，你穷尽了所有四种可能性。你得到了所有四种可能的状态。在这里，我们做了一个简单的调整：我们应用了哈达玛门，然后不是对下一个量子比特应用另一个哈达玛门，而是应用了一个 CNOT 门。

Entanglement and the Bell Circuit

In our previous circuit with two qubits and a **Hadamard gate**, we exhausted all four possible output states. Here, we introduce a simple twist. Instead of applying another Hadamard gate to the second qubit, we apply a **CNOT gate** (Conditional NOT gate) and then measure.

This simple circuit is an **entanglement circuit**. It entangles qubits A and B. If you measure the first qubit and get a 0, the second qubit will automatically be 0. Conversely, if you measure a 1, the second qubit will be 1. You will always observe either `|00>` or `|11>` as the output.

This specific circuit is called the **Bell circuit**. Initially, there were four possible states. What this circuit does is curtail two of those possibilities: `|01>` and `|10>` will never appear. They are eliminated from the output.

Analyzing the Bell State

Let's analyze this using another approach for designing quantum algorithms. We start with two qubits in the initial state `|00>`. After applying the Bell circuit, the final state is either `|00>` or `|11>`.

The possible basis states are `|00>`, `|01>`, `|10>`, and `|11>`. For the output `|00>`, what should its probability amplitude be? The probability of measuring `|00>` is 1/2. Therefore, the corresponding amplitude is `1/√2`.

So, the state can be represented as:

```
(1/√2)|00> + ...
```

Another approach is to start from the initial state and determine how to reach this final entangled state.

量子纠缠与贝尔电路

在我们之前有两个量子比特和一个**哈达玛门**的电路中，我们穷尽了所有四种可能的输出状态。这里，我们引入一个简单的调整。我们不对第二个量子比特应用另一个哈达玛门，而是应用一个**CNOT门**（受控非门），然后进行测量。

这个简单的电路是一个**纠缠电路**。它使量子比特A和B发生纠缠。如果你测量第一个量子比特得到0，第二个量子比特将自动为0。反之，如果测量得到1，第二个量子比特将为1。你将总是观察到`|00>`或`|11>`作为输出。

这个特定的电路被称为**贝尔电路**。最初，存在四种可能的状态。这个电路的作用是削减其中两种可能性：`|01>`和`|10>`永远不会出现。它们从输出中被消除了。

分析贝尔态

让我们使用另一种设计量子算法的方法来分析。我们从两个处于初始状态`|00>`的量子比特开始。应用贝尔电路后，最终状态是`|00>`或`|11>`。

可能的基态是`|00>`、`|01>`、`|10>`和`|11>`。对于输出`|00>`，它的概率幅应该是多少？测量到`|00>`的概率是1/2。因此，相应的幅值是`1/√2`。

所以，该状态可以表示为：

```
(1/√2)|00> + ...
```

另一种方法是从初始状态开始，确定如何达到这个最终的纠缠态。

Circuit Design Philosophy & Student Interaction

The probability of measuring `|00>` is 1/2. Therefore, its corresponding **probability amplitude** is `1/√2`.

So, the state can be represented as `(1/√2)|00> + …`. Another approach to circuit design is to start from the initial state and determine how to reach the desired final state.

You must design the circuit to reduce the probability of undesired outputs and increase the amplitude of the desired outputs. In this case, we are designing for `|00>` and `|11>`. The goal is to create a circuit that amplifies the probabilities of expected states and suppresses those of unexpected ones. This perspective is fundamental to designing quantum circuits effectively.

You begin with a fixed initial state. From all possible output states, you identify the desired ones. The circuit must then be engineered to "shoot up" the probabilities of the expected states and "shoot down" the probabilities of the unexpected ones. This is precisely what the Bell state circuit accomplishes.

Initially, we are in a specific state. The expected outputs from the circuit are among the four possibilities: `|00>`, `|01>`, `|10>`, and `|11>`. Out of these four, we need to amplify the probabilities of `|00>` and `|11>` while suppressing the probabilities of `|01>` and `|10>`. The

design should suppress the undesired amplitudes to nearly zero and increase the desired ones to `1/√2`. This is the core methodology for this circuit design.

> 🙋‍♂️ **Student**: How is it better compared to the Hadamard gate? 🤦‍♂️ **Mohammad**: This is an entirely different solution. This is a **quantum solution for entanglement**. If you have two qubits and need to entangle them, this (the Bell circuit) is the solution.

> 🙋‍♂️ **Student**: So basically we are interested in only two outcomes. 🤦‍♂️ **Mohammad**: Yes, we are interested only in these two outcomes: `|00>` and `|11>`.

> 🙋‍♂️ **Student**: And if I take one qubit, using the Hadamard gate, we get only superposition.
> 🤦‍♂️ **Mohammad**: Yes, only superposition. So the problem was with a single qubit needing superposition. Then the **Hadamard gate** is the solution. We are understanding solutions problem-wise: the problem was superposition, so the Hadamard gate comes into play, and we designed a circuit accordingly.

电路设计理念与学生互动

测量到 `|00>` 的概率是 1/2。因此，其对应的**概率幅**为 `1/√2`。

所以，该状态可以表示为 `(1/√2)|00> + …`。电路设计的另一种方法是，从初始状态开始，确定如何达到期望的最终状态。

你必须设计电路以降低非期望输出的概率，并增加期望输出的幅值。在本例中，我们设计的目标是 `|00>` 和 `|11>`。目标是创建一个能放大期望态概率并抑制非期望态概率的电路。这个视角对于有效设计量子电路至关重要。

你从一个固定的初始状态开始。从所有可能的输出状态中，你识别出期望的状态。然后，电路必须被设计成"提升"期望态的概率并"压低"非期望态的概率。这正是贝尔态电路所实现的功能。

初始时，我们处于一个特定状态。电路的期望输出是四种可能性之一：`|00>`、`|01>`、`|10>` 和 `|11>`。在这四种之中，我们需要放大 `|00>` 和 `|11>` 的概率，同时抑制 `|01>` 和 `|10>` 的概率。设计应将非期望的幅值压制到接近零，并将期望的幅值增加到 `1/√2`。这是该电路设计的核心方法。

> 🙋‍♂️ **学生**：与哈达玛门相比，它（贝尔电路）更好在哪里？ 🤦‍♂️ **Mohammad**：这是一个完全不同的解决方案。这是一个**用于量子纠缠的解决方案**。如果你有两个量子比特并且需要纠缠它们，这个（贝尔电路）就是解决方案。

🙋 **学生**：所以我们基本上只对两种结果感兴趣。🤦 **Mohammad**：是的，我们只对这两种结果感兴趣：`|00>` 和 `|11>`。

🙋 **学生**：如果我取一个量子比特，使用哈达玛门，我们只能得到叠加态。🤦 **Mohammad**：是的，只能得到叠加态。所以问题在于单个量子比特需要叠加态。那么**哈达玛门**就是解决方案。我们是根据问题来理解解决方案的：问题是叠加，所以哈达玛门就派上用场了，我们据此设计电路。

Designing Circuits for Superposition and Entanglement

Mohammad confirms that using a **Hadamard gate** on a single qubit yields superposition. The problem was creating superposition for a single qubit, and the Hadamard gate is the solution. We understand the solution based on the problem: the problem was superposition, so the Hadamard gate is used to design the circuit.

For two qubits, we need to superpose both. We apply the **Hadamard gate** on both qubits to achieve a superposition of all possible states. The new problem is **entanglement**: if the first qubit is `0`, the second must automatically be `0`. We need to design a circuit for that.

The Entanglement Solution

The solution involves a **control node** and applying a **Hadamard gate** in a specific sequence. This is logically easy to understand. If the first qubit is `0`, why is the second `0`? Suppose you apply the Hadamard gate and measure, getting `0`. It automatically becomes `0` because `0` is the condition bit.

When the condition bit is `0` and the input is `0`, the output is obviously also `0`. The gate is dysfunctional and will not operate, resulting in `|00>`. The moment it becomes `1`, the condition bit is set to `1`, making the gate functional. Whatever `0` is gets converted to `1`, so the output becomes `|11>`.

🙋 **Student**: And if I take one qubit, using the Hadamard gate, we get only superposition.
🤦 **Mohammad**: Yes, only superposition. So then the problem was that single qubit and I need to superposition them. Then the Hadamard gate is the solution. So we are understanding solution problem wise. So problem was superposition, so Hadamard gate comes into place, so we designed a circuit simply using Hadamard gate.

This is how you logically design the circuit: if it's `0`, it's always `0`; if it's `1`, it's always `1`.

Modifying the Circuit for Flipped Entanglement

Now, if you need to flip it—so that if it's `0`, it should be `1`, and if it's `1`, it should be `0`—you must entangle them differently. What changes does this circuit need? Where should you add another **NOT gate (X gate)**?

- Should it be after the first Hadamard gate?
- Or should you apply an **X gate** on qubit `b`? Where exactly on `b`?
- Should it be on `b` after the **NOT gate**?
- Or on `b` before applying the **controlled gate (c0)**? Would applying an **X gate** before `c0` solve the problem?
- Or perhaps after the **c0 gate**?

Let's examine this. There's another interesting aspect to learn from this script.

Naming Quantum and Classical Registers

If you want to name your classical registers and qubits, here's another step. For that, you need to import `QuantumRegister` and `ClassicalRegister`. These two additional imports are required.

```
# Example of initializing named registers
from qiskit import QuantumRegister, ClassicalRegister

# Initialize qubits with names
qubit_a = QuantumRegister(1, 'A')  # Single qubit named 'A'
qubit_b = QuantumRegister(1, 'B')  # Single qubit named 'B'

# Initialize classical bits with names
classical_a = ClassicalRegister(1, 'AC')  # Classical bit named 'AC'
classical_b = ClassicalRegister(1, 'BC')  # Classical bit named 'BC'
```

In this way, you can also initialize your quantum and classical registers with specific names for better circuit management and readability.

设计叠加与纠缠的电路

Mohammad 确认，对单个量子比特使用**哈达玛门**会产生叠加态。问题在于为单个量子比特创建叠加态，而哈达玛门就是解决方案。我们根据问题来理解解决方案：问题是叠加，所以哈达

玛门就派上用场了，我们据此设计电路。

对于两个量子比特，我们需要使两者都处于叠加态。我们在两个量子比特上都应用**哈达玛门**，以获得所有可能状态的叠加。新问题是**纠缠**：如果第一个量子比特是`0`，第二个必须自动是`0`。我们需要为此设计一个电路。

纠缠的解决方案

该解决方案涉及一个**控制节点**和按特定顺序应用**哈达玛门**。这在逻辑上很容易理解。如果第一个量子比特是`0`，为什么第二个也是`0`？假设你应用哈达玛门并进行测量，得到`0`。它会自动变成`0`，因为`0`是条件位。

当条件位是`0`且输入是`0`时，输出显然也是`0`。该门功能失效，不会操作，从而得到`|00>`。当它变为`1`时，条件位被设为`1`，使该门功能生效。无论`0`是什么，都会被转换为`1`，因此输出变为`|11>`。

> 🙋‍♂️ **学生**：如果我取一个量子比特，使用哈达玛门，我们只能得到叠加态。 🤦‍♂️ **Mohammad**：是的，只能得到叠加态。所以问题在于单个量子比特需要叠加态。那么**哈达玛门**就是解决方案。我们是根据问题来理解解决方案的：问题是叠加，所以哈达玛门就派上用场了，我们据此设计电路。

这就是你逻辑上设计电路的方式：如果是`0`，则始终为`0`；如果是`1`，则始终为`1`。

修改电路以实现翻转的纠缠

现在，如果你需要翻转它——使得如果是`0`，则应变为`1`；如果是`1`，则应变为`0`——你必须以不同的方式使它们纠缠。这个电路需要什么改变？你应该在哪里添加另一个**非门（X门）**？

- 是在第一个哈达玛门之后吗？
- 还是应该在量子比特`b`上应用一个**X门**？具体在`b`的哪个位置？
- 是在**非门**之后的`b`上吗？
- 还是在应用**受控门（c0）**之前的`b`上？在`c0`之前应用**X门**能解决问题吗？
- 或者也许在**c0门**之后？

让我们来研究一下。从这个脚本中还可以学到另一个有趣的方面。

命名量子寄存器和经典寄存器

如果你想为你的经典寄存器和量子比特命名，这是另一个步骤。为此，你需要导入 `QuantumRegister` 和 `ClassicalRegister`。需要这两个额外的导入。

```
# 初始化命名寄存器的示例
from qiskit import QuantumRegister, ClassicalRegister

# 使用名称初始化量子比特
qubit_a = QuantumRegister(1, 'A')  # 名为 'A' 的单个量子比特
qubit_b = QuantumRegister(1, 'B')  # 名为 'B' 的单个量子比特

# 使用名称初始化经典比特
classical_a = ClassicalRegister(1, 'AC')  # 名为 'AC' 的经典比特
classical_b = ClassicalRegister(1, 'BC')  # 名为 'BC' 的经典比特
```

通过这种方式，你也可以用特定的名称初始化你的量子和经典寄存器，以便更好地管理和提高电路的可读性。

Initializing Quantum Circuits with Named Registers

To initialize a circuit with named registers, you need to import the `ClassicalRegister` class. You can then create and name your quantum and classical registers individually.

```
# Initialize named quantum registers (single qubit each)
quantum_a = QuantumRegister(1, 'A')  # Single qubit named 'A'
quantum_b = QuantumRegister(1, 'B')  # Single qubit named 'B'

# Initialize named classical registers (single bit each)
classical_ac = ClassicalRegister(1, 'AC')  # Classical bit named 'AC'
classical_bc = ClassicalRegister(1, 'BC')  # Classical bit named 'BC'

# Create a quantum circuit using the named registers
circuit = QuantumCircuit(quantum_a, quantum_b, classical_ac, classical_bc)
```

This method provides more sophistication and clarity compared to the simpler `QuantumCircuit(2, 2)` initialization, where the first argument (`2`) specifies the number of quantum bits and the second (`2`) specifies the number of classical bits. Naming your registers makes the circuit's purpose and structure much easier to understand.

Applying Gates and Measurement

Once the circuit is initialized, you can apply gates. The **Hadamard (H) gate** is applied to a specific qubit (e.g., `quantum_a[0]`). The **CNOT (CX) gate** takes two arguments:

1. The first argument is the **control qubit**.
2. The second argument is the **target qubit** that gets flipped conditionally.

Finally, measurements are performed, mapping each qubit's state to a specific classical bit for readout (e.g., `quantum_a[0]` to `classical_ac[0]`).

Circuit Execution and Results

When this circuit is executed with 4096 shots, the results are approximately 2048 counts for the `00` state and 2048 counts for the `11` state. Minor statistical variations (e.g., 2043 and 2053) are expected. The key observation is the absence of the `01` and `10` states.

> 🙋 Student: Why do we mainly see 0,0 and 1,1 but not 0,1 and 1,0? What in this circuit prevents 0,1 and 1,0 from appearing? 👷 Lecturer: I will explain what exactly happens. The circuit creates entanglement. If the control qubit is in state |0>, the CNOT gate becomes inactive, and the target qubit remains unchanged. Conversely, if the control qubit is |1>, it flips the target qubit. This correlation, combined with the initial Hadamard gate creating superposition on the control, results in the **Bell state** (|00> + |11>)/√2, which only yields correlated `00` or `11` outcomes upon measurement, never the anti-correlated `01` or `10`.

使用命名寄存器初始化量子电路

要使用命名寄存器初始化电路，你需要导入 `ClassicalRegister` 类。然后，你可以单独创建并命名你的量子和经典寄存器。

```
# 初始化命名的量子寄存器（每个为单个量子比特）
quantum_a = QuantumRegister(1, 'A')  # 名为 'A' 的单个量子比特
quantum_b = QuantumRegister(1, 'B')  # 名为 'B' 的单个量子比特

# 初始化命名的经典寄存器（每个为单个经典比特）
classical_ac = ClassicalRegister(1, 'AC')  # 名为 'AC' 的经典比特
classical_bc = ClassicalRegister(1, 'BC')  # 名为 'BC' 的经典比特

# 使用命名的寄存器创建量子电路
circuit = QuantumCircuit(quantum_a, quantum_b, classical_ac, classical_bc)
```

与更简单的 `QuantumCircuit(2, 2)` 初始化方法相比，此方法提供了更高的精细度和清晰度。在简单方法中，第一个参数 (`2`) 指定量子比特的数量，第二个参数 (`2`) 指定经典比特的数量。为寄存器命名使电路的目的和结构更容易理解。

应用门和测量

电路初始化后，即可应用量子门。**Hadamard (H) 门** 被应用于特定的量子比特（例如 `quantum_a[0]`）。**CNOT (CX) 门** 接受两个参数：

1. 第一个参数是 **控制量子比特**。
2. 第二个参数是 **目标量子比特**，它根据条件被翻转。

最后，执行测量，将每个量子比特的状态映射到特定的经典比特以供读出（例如，将 `quantum_a[0]` 映射到 `classical_ac[0]`）。

电路执行与结果

当此电路以 4096 次采样执行时，结果大致为 `00` 态 2048 次计数和 `11` 态 2048 次计数。微小的统计波动（例如 2043 和 2053）是正常的。关键的观察结果是 `01` 和 `10` 态的缺失。

> 🙋 学生：为什么我们主要看到 0,0 和 1,1，而没有 0,1 和 1,0？这个电路中是什么阻止了 0,1 和 1,0 的出现？ 🤦 讲师：我来解释一下到底发生了什么。这个电路创建了纠缠。如果控制量子比特处于 |0> 态，CNOT 门将不激活，目标量子比特保持不变。相反，如果控制量子比特是 |1>，它将翻转目标量子比特。这种关联性，加上初始 Hadamard 门在控制比特上创建的叠加态，导致了 **贝尔态** (|00> + |11>)/√2 的产生。该态在测量时只会产生相关的 `00` 或 `11` 结果，而永远不会产生反相关的 `01` 或 `10` 结果。

Q&A: Understanding the Bell State Circuit

> 🙋 Student: Why do we mainly see 0,0 and 1,1 but not 0,1 and 1,0? 🤦 Lecturer: The circuit creates a specific entangled state that forbids the anti-correlated outcomes. Let me explain the mechanism.

If the control qubit (the conditional bit) is `0`, the **CNOT (CX) gate** is inactive. The target qubit's initial input (e.g., `0`) passes through unchanged. To get a `1` on the target, you would need to apply an **X gate** to it beforehand.

Conversely, if the control qubit is `1`, the CNOT gate activates. If the target's input is `1`, the **NOT operation** flips it to `0`. Therefore, the **X gate** applied *before* the CNOT is crucial for preparing the desired input state for the target qubit.

Applying an X gate *after* the CNOT would likely yield a different result. The standard **Bell circuit** configuration applies the X gate *before* the CNOT. The core role of the **CX gate** is to **entangle** both qubits.

Practical Exercises on Circuit Modification

The lecturer assigned the following exercises to be completed before the next class:

1. Modify the circuit: Remove the **CX gate** and apply an **X gate** to qubit `V` just before measurement.
2. Apply a **Z gate** to qubit `A` *before* the **CX gate**.
3. For both modifications, observe and describe the changes in the resulting histogram.

Students were given the option to start these exercises immediately or complete them later, as the lecture needed to proceed.

Introduction to the Bloch Sphere for Visualization

Designing quantum algorithms can be complex and sometimes counterintuitive, as we are working at the hardware circuit level, not with straightforward classical logic. To aid understanding, **visualization tools** are essential.

These tools allow you to visualize **state vectors** and results, making it easier to navigate the quantum computing environment. For visualizing the state of a **single qubit**, the primary tool is the **Bloch sphere**.

The Bloch sphere is a geometrical representation. Everyone is familiar with the concept of a sphere—a three-dimensional ball. This sphere provides a complete picture of a single qubit's quantum state.

Bloch Sphere Visualization

Mohammad explains that the **Bloch sphere** is a tool for visualizing a single qubit. Everyone is familiar with the concept of a sphere—a three-dimensional ball. This sphere provides a complete picture of a single qubit's quantum state.

The Z-Axis and Basis States

Imagine the center of the sphere. The vertical axis is the **Z-axis**. The direction pointing upwards is the **positive Z-axis**, and the direction pointing downwards is the **negative Z-axis**.

On this primary axis, there is a key mapping:

- The **positive Z-axis** corresponds to the `|0⟩` vector (the **zero state**).
- The **negative Z-axis** corresponds to the `|1⟩` vector (the **one state**).

These two points are diametrically opposite because the `|0⟩` and `|1⟩` states are orthogonal basis vectors.

Superposition and State Space

All other qubit states are a **superposition** of `|0⟩` and `|1⟩`. This superposition is defined by the complex probability amplitudes, **alpha (α)** and **beta (β)**. The entire surface of the Bloch sphere represents the space where the state vector of a single qubit can reside.

From the center, the vector can point to any location on the sphere's surface. This illustrates the vast number of possible states for just a single qubit.

Implication for Quantum Computing

This immense state space is one reason why the quantum world offers such powerful representation. With a minimum number of qubits, you can represent a vast amount of information due to this inherent **parallelism**.

A single qubit has infinite possible states on the sphere, allowing it to represent a lot of information. Quantum computing exploits this massive parallelism to solve complex problems efficiently. We will revisit this concept later; for now, the focus is on understanding the visualization.

布洛赫球可视化

Mohammad 解释说，**布洛赫球** 是用于可视化单个量子比特的工具。每个人都熟悉球体的概念——一个三维的球。这个球体提供了一个单量子比特量子态的完整图像。

Z轴与基态

想象球体的中心。垂直轴是 **Z轴**。向上的方向是 **正Z轴**，向下的方向是 **负Z轴**。

在这个主轴上，有一个关键的映射：

- **正Z轴** 对应于 `|0⟩` 矢量（**零态**）。
- **负Z轴** 对应于 `|1⟩` 矢量（**一态**）。

这两个点正好相对，因为 `|0⟩` 和 `|1⟩` 态是正交的基矢量。

叠加态与态空间

所有其他量子比特态都是 `|0⟩` 和 `|1⟩` 的 **叠加**。这种叠加由复概率幅 **alpha (α)** 和 **beta (β)** 定义。布洛赫球的整个表面代表了单个量子比特的状态矢量可以存在的空间。

从中心出发，矢量可以指向球体表面的任何位置。这说明了仅仅一个量子比特就有大量可能的状态。

对量子计算的意义

这个巨大的态空间是量子世界能够提供如此强大表示能力的原因之一。由于这种固有的 **并行性**，用最少数量的量子比特就可以表示海量的信息。

单个量子比特在球体上有无限可能的状态，使其能够表示大量信息。量子计算利用这种大规模的并行性来高效解决复杂问题。我们稍后会重新讨论这个概念；目前，重点是理解可视化。

布洛赫球面坐标轴与基础向量

单个量子比特可以表示大量信息，这正是其意义所在——它为计算带来了巨大的**并行性**。量子计算正是利用这一点来解决众多复杂问题。我们稍后会重新讨论这个概念；目前，我们专注于理解其可视化。

在布洛赫球面中：

- **正 Z 轴** 代表状态 |0⟩。
- **负 Z 轴** 代表状态 |1⟩。
- 朝向观察者的方向是 **正 X 轴**。
- 背离观察者的方向是 **负 X 轴**。
- 向左是 **正 Y 轴**。

- 向右是 **负 Y 轴**。

任何量子态都是这些基础向量的组合。我们使用 Z、X、Y 这三个轴来划分这个空间。这也解释了为什么我们之前看到的 **X 门**、**Y 门**、**Z 门** 与这些坐标轴存在关联。

表示量子态的两种坐标系

为了在布洛赫球面上唯一地表示一个向量，我们需要知道它的坐标。有两种主要的坐标系：

1. **笛卡尔坐标系**：需要知道向量的 X、Y、Z 三个坐标值。这在布洛赫球面中可行，但在实际量子操作中并不实用，因为我们通常不直接处理这些坐标值。
2. **球面坐标系**：这是一种更实用的方法，使用三个参数来描述同一个向量：
   - **长度 (r)**：向量从球心出发的长度。
   - **极角 (θ)**：向量与正 Z 轴之间的夹角。
   - （第三个参数——方位角 φ——将在后续说明。）

布洛赫球坐标轴与基矢

单个量子比特可以表示大量信息，这正是其意义所在——它为计算带来了巨大的**并行性**。量子计算正是利用这一点来解决众多复杂问题。我们稍后会重新讨论这个概念；目前，我们专注于理解其可视化。

在布洛赫球面中：

- **正 Z 轴** 代表状态 |0⟩。
- **负 Z 轴** 代表状态 |1⟩。
- 朝向观察者的方向是 **正 X 轴**。
- 背离观察者的方向是 **负 X 轴**。
- 向左是 **正 Y 轴**。
- 向右是 **负 Y 轴**。

任何量子态都是这些基矢的组合。我们使用 Z、X、Y 这三个轴来划分这个空间。这也解释了为什么我们之前看到的 **X 门**、**Y 门**、**Z 门** 与这些坐标轴存在关联。

表示量子态的两种坐标系

为了在布洛赫球面上唯一地表示一个向量，我们需要知道它的坐标。有两种主要的坐标系：

1. **笛卡尔坐标系**：需要知道向量的 X、Y、Z 三个坐标值。这在布洛赫球面中可行，但在实际量子操作中并不实用，因为我们通常不直接处理这些坐标值。
2. **球面坐标系**：这是一种更实用的方法，使用三个参数来描述同一个向量：
   - **长度 (r)**：向量从球心出发的长度。
   - **极角 (θ)**：向量与正 Z 轴之间的夹角。
   - （第三个参数——方位角 φ——将在后续说明。）

Describing a Vector on the Bloch Sphere: Polar Angle

So, what is the length of a vector? That length is denoted by **r**. To draw a vector from the center of the sphere, I need to know how far to go, which is its length.

The second parameter I need is the angle this vector makes with the positive Z-axis. This is called the **polar angle**, often denoted by **θ**. For example, if I know this angle is 60 degrees, I can start drawing the vector from that marked angle.

Why is it Called the Polar Angle?

This is called the polar angle because the Z-axis is the **polar axis**, with the north and south poles. This angle measures how much the vector is deflected from this polar axis. The value of this angle gives a hint about the qubit's state.

- If the vector is near the **north pole** (polar angle near 0°), it is closer to the state |0⟩.
- If it is more towards the **south pole** (polar angle near 180°), it is closer to the state |1⟩.
- Consequently, the magnitude (probability amplitude) for |0⟩ (`alpha`) will be larger when the vector is near the north pole.

Visual Interpretation and Classical Limit

Through visualization, if a vector is very close to the north pole (polar axis), the probability is high that a measurement will cause it to **collapse** to |0⟩. To collapse to |1⟩, it would have to deviate significantly.

This explains the classical states:

- A classical bit `0` is represented by a vector exactly on top of the north pole. When measured, it will always collapse to |0⟩.
- A classical bit `1` is represented by a vector exactly on the south pole.

This demonstrates that **classical physics and classical computation are just a special case of quantum physics and quantum computing**. You can represent classical physics using quantum theory, but not the reverse.

> 👩‍🏫 Lecturer: Everyone understood what the polar angle is? Can you tell me what could be the range of this polar angle? 🙋‍♂️ Student: 180. 👩‍🏫 Lecturer: Yes, the maximum it can go is 180 degrees. What is the minimum? 🙋‍♂️ Student: Zero. 👩‍🏫 Lecturer: So it can range from zero to 180 degrees.

在布洛赫球面上描述向量：极角

那么，向量的长度是多少？这个长度用 **r** 表示。为了从球心画一个向量，我需要知道需要走多远，这就是它的长度。

我需要的第二个参数是这个向量与正 Z 轴所成的角度。这被称为 **极角**，通常用 **θ** 表示。例如，如果我知道这个角是 60 度，我就可以从那个标记的角度开始画这个向量。

为什么它被称为极角?

这被称为极角，因为 Z 轴是 **极轴**，有北极和南极。这个角度衡量了向量偏离这个极轴的程度。这个角度的值暗示了量子比特的状态。

- 如果向量靠近 **北极**（极角接近 0°），则它更接近状态 |0⟩。
- 如果它更朝向 **南极**（极角接近 180°），则它更接近状态 |1⟩。
- 因此，当向量靠近北极时，|0⟩ 的幅度（概率幅 `alpha`）会更大。

视觉解释与经典极限

通过可视化，如果一个向量非常靠近北极（极轴），那么测量导致它 **坍缩** 到 |0⟩ 的概率就很高。要坍缩到 |1⟩，它必须显著偏离。

这解释了经典状态：

- 经典比特 `0` 由正好在北极顶端的向量表示。测量时，它总是坍缩到 |0⟩。
- 经典比特 `1` 由正好在南极的向量表示。

这表明 **经典物理学和经典计算只是量子物理学和量子计算的一个特例**。你可以用量子理论表示经典物理学，但反之则不行。

> 👩‍🏫 讲师：大家都明白极角是什么了吗？你们能告诉我这个极角的范围可能是多少吗？ 🙋‍♂️ 学生：180。 👩‍🏫 讲师：是的，最大可以达到 180 度。最小值是多少？ 🙋‍♂️ 学生：零。 👩‍🏫 讲师：所以它的范围可以从零到 180 度。

Defining a Vector in 3D Space

To correctly define a vector in three-dimensional space, two angles are required in addition to its length. The first is the **polar angle** (θ), which we previously established ranges from 0 to 180 degrees.

The second crucial angle is the **azimuthal angle** (φ). Imagine the vector's projection onto the x-y plane, like a shadow cast by a light source from above. The angle this projection makes with the positive x-axis is the azimuthal angle, denoted by φ.

Angles and Dimensions

In summary, to uniquely specify a vector's direction:

- In **2D**, you need **one angle** and a length.
- In **3D**, you need **two angles** (θ and φ) and a length.
- In **4D**, you would need **three angles** and a length.

This pattern continues into higher dimensions (nD requires n-1 angles), which, while difficult to visualize, is a valid conceptual framework. This principle of using angles to define a state is foundational to the **Bloch sphere**.

> 👩‍🏫 Lecturer: Can you see? This is positive Z, representing the |0⟩ state. And if you look at its state vector…

在三维空间中定义向量

为了在三维空间中正确定义一个向量，除了其长度外，还需要两个角度。第一个是**极角** (θ)，我们之前已确定其范围是0到180度。

第二个关键角度是**方位角** (φ)。想象一下向量在x-y平面上的投影，就像从上方光源投射下的阴影。这个投影与正x轴所成的角度就是方位角，记作φ。

角度与维度

总结来说，要唯一确定一个向量的方向：

- 在**二维**中，需要**一个角度**和长度。
- 在**三维**中，需要**两个角度** (θ 和 φ) 和长度。
- 在**四维**中，则需要**三个角度**和长度。

这种模式可以延续到更高维度（n维需要n-1个角度），虽然难以可视化，但这是一个有效的概念框架。这种用角度定义状态的原则是**布洛赫球**的基础。

> 👩‍🏫 讲师：大家能看到吗？这是正Z轴，代表 |0⟩ 态。如果你看它的态矢量…

Visualizing Quantum States on the Bloch Sphere

Mohammad concludes the explanation of the **Bloch sphere** and transitions to a live demonstration using code. He shows how to represent the |0⟩ and |1⟩ states and introduces the concept of quantum gates as rotations.

Demonstrating the |0⟩ and |1⟩ States

Here is the **Bloch sphere** which represents the |0⟩ state. It is located at the positive Z-axis. I am printing the **state vector** from my circuit here as it is.

If you notice, the state vector is [1, 0]. Don't worry about the complex number notation (`j`) for now; right now it's a straightforward number, 1. This is because the state is |0⟩. So, the first element is 1, and every other element is 0.

In the next class, I will walk you through the necessary mathematics, like what a **complex number** is and how we perform operations. This is the state vector, and it corresponds to the positive Z-axis. Similarly, the next state corresponds to |1⟩.

Introducing Quantum Gates as Rotations

What I am doing is initializing the qubit and then applying the **X-gate** to get the |1⟩ state. Then I am writing its state vector and plotting it. This circuit initializes the qubit and then applies the X gate. Over time, I will walk you through gates as well. **Qiskit** is the right tool to visualize what gates are doing.

If you see, I am applying an X gate. Before applying the X gate in my last cell, the vector was [1, 0]. Now when I apply the X gate, my vector becomes [0, 1]. So, what exactly is the **X gate** doing? The X gate is rotating the state vector. It performs a **rotation of 180 degrees** with respect to a specific axis.

> 👨‍🏫 Mohammad: With respect to which axis? 🙋‍♂️ Speaker 2: The X-axis? 👨‍🏫 Mohammad: The X gate rotates with respect to the **X-axis**. The Y gate rotates with respect to the **Y-axis**. The Z gate rotates with respect to the **Z-axis**.

Understanding the Axis of Rotation

For rotation, you need some axis to rotate. Rotation means, suppose this is my state, then I'm rotating it. What is my **axis of rotation**? Can somebody tell me? Not in X, Y, Z terms, just tell me in layman terms—what is my axis of rotation? This is static; this is the hinge. Against that hinge, I'm rotating.

在布洛赫球上可视化量子态

Mohammad 总结了**布洛赫球**的解释，并过渡到使用代码进行实时演示。他展示了如何表示 |0⟩ 和 |1⟩ 态，并介绍了量子门作为旋转的概念。

演示 |0⟩ 和 |1⟩ 态

这就是**布洛赫球**，它代表 |0⟩ 态。它位于正Z轴上。我在这里直接打印我电路中的**态矢量**。

如果你注意到，态矢量是 [1, 0]。现在先不要管复数表示法（`j`）；目前它是一个简单的数字 1。这是因为态是 |0⟩。所以，第一个元素是 1，其他所有元素都是 0。

在下节课中，我将带你们学习必要的数学知识，比如什么是**复数**以及我们如何进行运算。这就是态矢量，它对应正Z轴。类似地，下一个态对应 |1⟩。

介绍作为旋转的量子门

我正在做的是初始化量子比特，然后应用 **X门** 来得到 |1⟩ 态。然后我写出它的态矢量并绘图。这个电路初始化量子比特，然后应用 X 门。随着时间的推移，我也会带你们学习各种门。**Qiskit** 是可视化门作用的合适工具。

如果你看，我正在应用一个 X 门。在我上一个单元格中应用 X 门之前，矢量是 [1, 0]。现在当我应用 X 门时，我的矢量变成了 [0, 1]。那么，**X门** 到底在做什么？X 门正在旋转态矢量。它执行**绕特定轴旋转180度**的操作。

> 🤦 Mohammad：绕哪个轴旋转？ 🤷 Speaker 2：X轴？ 🤦 Mohammad：X 门绕 **X轴** 旋转。Y 门绕 **Y轴** 旋转。Z 门绕 **Z轴** 旋转。

理解旋转轴

对于旋转，你需要一个轴来绕其旋转。旋转意味着，假设这是我的态，然后我旋转它。我的**旋转轴**是什么？有人能告诉我吗？不用 X, Y, Z 这些术语，就用通俗的话说——我的旋转轴是什么？这是静态的；这是铰链。我绕着那个铰链在旋转。

理解旋转轴与X、Y、Z门

假设这是我的量子态，我正在旋转它。那么我的**旋转轴**是什么？用通俗的话说，旋转轴就像一个静态的铰链，我绕着那个铰链在旋转。

- 当应用 **X门** 时，旋转的铰链是 **X轴**。这个门会使量子态矢量绕X轴逆时针旋转180度。
- 当应用 **Y门** 时，旋转轴是 **Y轴**，同样进行180度旋转。
- 当应用 **Z门** 时，旋转轴是 **Z轴**。

关于Z门的一个关键点

如果量子态矢量本身就位于Z轴之上，那么应用Z门会发生什么？

> 🤷 Speaker 2：只是移动它？ 🤦 Mohammad：如果你旋转一个与其旋转轴完全重合的物体，它只会原地旋转，始终保持在同一个轴上。因此，应用Z门后，矢量仍然会留在原地。

X、Y和Z门都是单量子比特门，它们都执行180度的旋转。唯一的区别在于它们所绕的**旋转轴**不同：X门绕X轴，Y门绕Y轴，Z门绕Z轴。关键在于，如果矢量正好位于旋转轴本身之上，旋转后它仍会停留在原处。

课后任务

在下节课之前，你们需要完成以下任务：

1. 完成课程中布置的所有活动。

2. 针对我在第一节课提出的问题——"构建量子比特的技术有哪些？"——撰写一份反思报告。

提交方式：

- 你可以将反思报告添加到同一文件的额外单元格中。
- 完成所有工作后，请将文件分享给我。
- 建议在GitHub上建立代码仓库来管理所有作业，并持续与我分享进展。

请务必完成这些任务，我需要了解大家的完成情况。谢谢，我们下次课见。

Understanding the Axis of Rotation and X, Y, Z Gates

Suppose this is my quantum state, and I'm rotating it. So what is my **axis of rotation**? In layman's terms, the axis of rotation is like a static hinge; I'm rotating against that hinge.

- When applying an **X gate**, the hinge is the **X-axis**. This gate rotates the state vector 180 degrees anticlockwise around the X-axis.
- When applying a **Y gate**, the axis is the **Y-axis**, also performing a 180-degree rotation.
- When applying a **Z gate**, the axis is the **Z-axis**.

A Key Point About the Z Gate

What happens if the quantum state vector is right on top of the Z-axis itself when applying the Z gate?

> 🙋‍♂️ Speaker 2: Just shift it? 🤦‍♂️ Mohammad: If you rotate something that lies exactly on its axis of rotation, it will just spin in place and always remain on the same axis. Therefore, after applying the Z gate, the vector remains there.

X, Y, and Z gates are all single-qubit gates. They all perform a 180-degree rotation. The only difference is the **axis of rotation** they act upon: the X gate rotates around the X-axis, the Y gate around the Y-axis, and the Z gate around the Z-axis. The key point is that if a vector is right on top of the rotation axis itself, it will remain there after rotation.

Post-Class Tasks

By the next class, you need to complete the following tasks:

1. Complete all the activities assigned during the lecture.
2. Write a reflection on the question I asked in the first class: "What are the technologies to build qubits?"

Submission Method:

- You can add your reflection to the same file in some new cells.
- After completing your work, you must share these files with me.
- It is recommended to build a repository on GitHub to keep all your work and share your progress with me continuously.

Please ensure you do this work, as I need to know how many of you have completed it. Alright, thank you. We'll see you next time.

Administrative Instructions

You must complete the assigned work by the next class. Please share the resulting files with me after you finish.

You can build and manage all your code in a GitHub repository. This is a good way to keep all your work organized and to share your progress with me continuously.

Please ensure you do this work, as I need to know how many of you have completed it. Alright, thank you. We'll see you next time.

Summary & Recommended Resources

Summary: This segment contains final administrative instructions for the course. The lecturer emphasizes the completion and submission of assigned work, suggests using GitHub for version control and progress sharing, and confirms the need for attendance tracking.

Recommended Level 9 Resources:

- **GitHub Official Documentation:** For mastering repository management, branching strategies, and collaboration workflows essential for professional and academic software projects.
- **Academic Papers on Software Engineering Practices:** Explore literature in journals like *IEEE Transactions on Software Engineering* on topics such as "Effective Use of Version Control in Distributed Teams" or "Tool Support for Continuous Integration in Academic Settings."

完整中文翻译

课程管理通知

你们必须在下次课前完成指定的作业。完成后，请将相关文件分享给我。

你们可以在 GitHub 上建立并管理所有的代码仓库。这是一个很好的方式来组织你的所有工作，并可以持续与我分享你的进展。

请务必完成此项工作，因为我需要了解有多少同学已经完成。好的，谢谢。我们下次课见。

总结与推荐资源

总结: 此部分为课程最终的管理性通知。讲师强调了完成并提交指定作业的要求，建议使用 GitHub 进行版本控制和进度共享，并确认了需要统计完成情况。

推荐 Level 9 资源:

- **GitHub 官方文档：** 用于掌握专业和学术软件项目所必需的仓库管理、分支策略和协作工作流。
- **软件工程实践相关的学术论文：** 查阅如 *IEEE Transactions on Software Engineering* 等期刊中的文献，主题可涉及"分布式团队中版本控制的有效使用"或"学术环境中持续集成的工具支持"。